

PhyReAct: Agentic Physical Reasoning for Video Evaluation and Refinement

Adobe Research

Generated videos can look convincing while violating prompt-specific physical obligations, yet clip-level learned scorers compress failures into opaque scores and offer little evidence for correction. We present **PhyReAct**, an agentic framework for auditable evaluation of generated video and a foundation for verification-guided refinement. **PhyReAct** compiles a prompt into prompt-grounded claims with typed checks and an executable measurement plan. A vision-language reasoner performs semantic decomposition and dense-frame judgments, while frozen open specialists supply counts, tracks, masks, depth, text, and audio evidence. The video’s timeline structures execution: occurrence gates timing, localized intervals select frames for later checks, and fixed rules compute relations. Specialists return measurements, not verdicts; deterministic rules combine validated records into *supported*, *contradicted*, or *unknown* claim states and a *plausible*, *implausible*, or *abstain* clip decision. This separation preserves provenance and keeps unresolved checks from becoming implicit successes. To ground evaluation in real failures, we introduce a 1,500-clip human-annotated corpus with prompt-linked flaw rationales and, where available, temporal spans and box tracks, from which we freeze a motion-enriched evaluation core. On 149 completed core clips, plan-based execution recovers 197 of 286 localized flaws (68.9%), versus 179 (62.6%) for one-at-a-time per-claim verification, at 13.0 versus 12.1 calls per clip. We also instantiate a simulation-conditioned generator as a controllable refinement testbed. **PhyReAct**’s evidence traces are designed to support future critic-guided refinement; closed-loop generator optimization remains future work.

Date: July 2026



1 Introduction

Text-to-video systems can now produce clips with striking appearance and smooth local motion, yet visual fluency is not physical reliability. A generated video may preserve style and identity while omitting a requested object, reversing the order of two events, violating a contact outcome, or producing a trajectory inconsistent with the described scene. Recent benchmarks expose this gap between perceptual quality, prompt adherence, and physical commonsense [5, 6, 36, 40]. For evaluation that can eventually support refinement, however, a scalar score is not enough. The system must recover the prompt-conditioned physical obligations—over entities, events, motion, contact, space, quantity, and time—and identify the evidence for each decision. This raises a first question: *can a video evaluator turn a generation request into checkable physical obligations and ground every decision in localized, inspectable evidence?*

Existing approaches provide complementary parts of this capability. Learned video judges estimate holistic quality or alignment scores [20, 21]; question and proposition decomposition make prompt requirements more explicit [9, 23]; and visual-programming or modular video-QA systems show how language models can compose perception modules [11, 18, 38, 51]. Physical verification nevertheless places these ideas under a demanding measurement contract. Counting objects, localizing an event, following a trajectory, estimating depth order, reading text, and detecting sound require different instruments and expose different failure modes. A high-level model must decide which evidence is needed and how later measurements depend on earlier ones, while low-level specialists must report what they observe without silently converting uncertain measurements into confident judgments. This leads to a second question: *can high-level reasoning coordinate heterogeneous measurements over time, preserve unresolved evidence, and produce feedback suitable for video refinement?*

To address these coupled questions, we present **PhyReAct**, an agentic physical reasoning framework for evaluating and refining generated video. Rather than treating physics as a single global impression, **PhyReAct**

compiles a prompt into verifiable claims and typed checks. Before inspecting the video, a high-level vision-language reasoner writes an executable plan whose operations and dependencies are checked by a mechanical validator. At execution time, a dense-frame reasoner establishes whether relevant objects and events occur, and frozen specialist models provide targeted measurements for count, tracking, segmentation, depth, rendered text, and audio. This division lets semantic reasoning select and connect the evidence while keeping specialist outputs available for inspection.

The central mechanism is a timeline-conditioned evidence chain. An event must first be observed before its time interval can support another operation; that interval then restricts the frames examined by a more expensive specialist; and temporal relations are evaluated from the resulting intervals using fixed rules. Consequently, evidence is not collected as an unordered set of model answers: earlier observations determine which later measurements are valid and where they should be made. Specialists return evidence rather than final prompt-satisfaction labels, after which deterministic composition maps each claim to **supported**, **contradicted**, or **unknown** and the clip to *plausible*, *implausible*, or **abstain**. The output is therefore a claim-level trace that identifies what was checked, where the evidence arose, and which conclusion followed from it.

Evaluating such a critic requires ground truth at the same level of detail. We therefore assemble 1,500 generated clips with 2,582 human-written flaw records. Each record identifies the exact prompt fragment that was violated and includes a rationale, severity, confidence, and, when available, a temporal span or object track. A recorded selection procedure yields a motion-enriched evaluation set and a 150-clip core for routine experiments. Unlike a single clip-level quality score, these annotations allow each allegation made by the critic to be matched to the particular failure marked by a person.

The same structured trace also provides a natural interface to refinement: contradicted claims identify what should change, while their localized evidence indicates when and where an intervention is needed. As a controlled testbed for this direction, we render MuJoCo simulations as depth or silhouette control videos for a Wan 2.2-VACE generator [25, 52, 53]. The simulation specifies geometry and motion, while the text prompt controls appearance. The present study evaluates the critic and this controllable generator separately; using critic feedback to update generation closes the intended loop and remains future work.

On the 149 development-core clips completed by both systems, **PhyReAct** recovers 197 of 286 localized flaws (68.9%), compared with 179 (62.6%) for wording-controlled, one-at-a-time claim verification. This gain of 18 recovered flaws has a whole-clip bootstrap interval of [+8, +28], while requiring 13.0 rather than 12.1 model and tool calls per clip. These results support planned, timeline-aware evidence collection over isolated per-claim verification at a modest additional call cost; because the core was used during system development, they characterize the current system rather than held-out generalization.

Our contributions are threefold. First, we introduce **PhyReAct**, an agentic critic that combines typed planning, heterogeneous visual and audiovisual measurements, timeline-conditioned execution, and deterministic three-valued reasoning in a single inspectable trace. Second, we provide a 1,500-clip benchmark with localized, prompt-grounded human flaw records and a paired flaw-level evaluation that isolates the benefit and cost of plan-based execution. Third, we connect structured criticism to a simulation-conditioned generation testbed, establishing a concrete interface for future critic-guided video refinement.

2 Physics-Guided Video Generation

2.1 Backbone and control mechanism

The backbone is Wan 2.2-VACE-Fun-14B, a two-expert mixture-of-experts video-diffusion model with a native control branch, run through the **WanVACEPipeline** interface; because the two expert stacks exceed the memory of a single 80 GB accelerator, the experts are CPU-offloaded during generation. Generation takes four inputs: the pre-computed control video, a mask, the text prompt, and one scalar conditioning strength. Nothing about appearance is taken from the simulation: the control video decides where surfaces are and how they move, while the prompt alone decides material, colour, lighting and setting (figure 1).

The mask decides which parts of the frame are held fixed and which are synthesised. Dark mask regions mark content to be preserved as given; bright regions mark content to be generated under the guidance of the

Bouncing ball

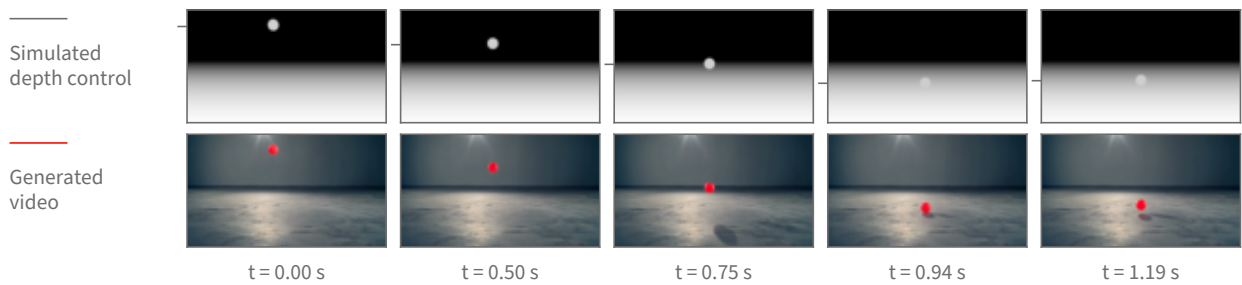


Figure 1 Physics-guided generation, shown on real frames of one bouncing-ball clip. **Top:** the depth control video rendered from the rigid-body simulation, the only geometric input the backbone receives. **Bottom:** the clip generated from it, where the text prompt supplies appearance alone. Both rows are sampled at the same five timestamps of the same 5.06 s, 16 fps, 848×480 clip; the timestamps are chosen by tracking the ball and taking the turning points of its vertical path, so the columns show the drop, the floor contact and the rebound rather than five evenly spaced instants. Tracking the ball independently in each row, the generated path follows the control’s with a correlation of 1.0000 horizontally and 0.9997 vertically over all 81 frames, and a median positional disagreement of 0.7 pixels across the frame and 4.7 pixels down. The tick beside each control frame marks the ball’s measured centre height; the frames themselves are unaltered.

control. Inside the backbone, the mask splits the control video into a preserved part and a generated part, which are encoded separately and concatenated into the control latent stream. We pass an *all-white* mask, so the preserved part is empty and every pixel of every frame is synthesised: the depth field is used purely as guidance and no region of it is copied through into the result. This is the mechanism that lets a grey depth render drive a photorealistic clip without tinting it grey.

The conditioning strength is the scalar by which the control latent stream is added into the denoising stream at each of the backbone’s control layers. One value applies uniformly to every control layer; a per-layer schedule is also accepted by the interface but is not used here. Turning it down weakens how tightly the generated motion is bound to the simulated motion without changing anything else about the request.

Appearance comes only from text. The prompt names the target scene in the ordinary way (“a bright red rubber ball bouncing on a polished concrete studio floor”), and a single fixed negative prompt—shared by every clip—pushes away from the rendered look the control video literally has, listing cartoon, CGI, render, grayscale and flat shading among the things to avoid. No appearance reference image is supplied. Sampling is deterministic given the seed, and the seed is recorded per clip.

Two properties of this arrangement are worth stating plainly. First, the backbone consumes a control video that was *computed*, not estimated: no monocular depth predictor sits in the loop, so the geometry the generator follows is exactly the simulator’s geometry, with no estimation error of its own. Second, this is a control-branch mechanism rather than image-to-image editing: it is neither an SDEdit-style noised re-synthesis [35] of a rendered source nor a copy of source pixels, which is what separates it from conditioning schemes that carry a synthetic source’s look into the output.

2.2 Rendering the control signal

One simulation, several registered views. A scene is described as data—a list of moving bodies with initial positions and velocities, a list of static surfaces, and a camera placed by a look-at rule—and the same engine turns any such description into a simulation. Dynamics are integrated in MuJoCo [52] under gravity at 9.81 m/s^2 with a one-millisecond solver step, a fourth-order integrator, and an elliptic friction cone. Output frames are produced on a coarser clock: the requested scene duration is divided into the frame budget, and the solver is advanced by that many sub-steps between frames. At every output frame, and from the *same* solver state, three renderers are queried in turn—a shaded colour view, the depth buffer, and the object-segmentation buffer. Because they read one state, the resulting videos are registered pixel for pixel, so a depth control and a silhouette control for the same scene describe the same geometry and can be substituted for one another

without re-simulating.

From depth buffer to control video. The depth buffer holds distance from the camera. It is turned into a control video by an inverted, clipped affine map: distances are mapped so that near surfaces are bright and far surfaces dark, which is the polarity monocular depth estimators produce and therefore the polarity the control branch was trained to read. The mapping’s endpoints are robust percentiles—the first and the ninety-seventh—of the depth values in the scene, computed after discarding the far-clip constant that the background sky writes into the buffer, so that a single distant constant cannot compress the whole range into a few grey levels. Formally, for depth frame D_t over pixel domain Ω and $t \in \{0, \dots, T-1\}$, let

$$\begin{aligned} \mathcal{D} &= \{D_t(u) : t \in \{0, \dots, T-1\}, u \in \Omega, D_t(u) \neq d_{\text{far}}\}, \\ d_- &= Q_{0.01}(\mathcal{D}), \quad d_+ = Q_{0.97}(\mathcal{D}), \\ C_t(u) &= 1 - \text{clip}\left(\frac{D_t(u) - d_-}{d_+ - d_-}, 0, 1\right). \end{aligned} \tag{1}$$

Here d_{far} is the renderer’s background constant and C_t is the grayscale control frame before boundary smoothing. If $d_+ = d_-$, the depth map is degenerate and the silhouette control described below is used instead.

Why the mapping is fixed for the whole clip. The endpoints are computed once over all frames of the clip, not per frame. This is a substantive choice, not a detail: it makes the assigned grey level a function of distance alone, so the same distance reads as the same grey in every frame, and a change in brightness therefore means a change in distance. If the endpoints were recomputed per frame—which is the convention monocular depth estimators follow, since they emit one relative map per image—the grey assigned to a distance would depend on the depth distribution of that particular frame. A body holding its distance while other content enters or leaves the view would then change brightness, and the control branch has no way to distinguish that from the body actually moving toward or away from the camera. Fixing the mapping across the clip removes that ambiguity by construction. A per-frame variant is still rendered alongside the fixed-mapping one so the two can be compared directly on the same simulation.

Boundary treatment. A simulator’s depth buffer has ideal, hard edges: a silhouette boundary is a one-pixel step. Estimated depth maps, which is what the control branch has seen, do not look like that—their boundaries are soft. Each grayscale frame therefore receives a small symmetric Gaussian blur before encoding. Because the kernel is symmetric, the half-intensity contour of an edge stays where it was, so the smoothing changes how the boundary is presented without displacing the geometry it represents.

Silhouettes, and when they replace depth. The segmentation buffer labels each pixel with the geometry that produced it. Collecting the labels that belong to the simulation’s moving bodies and painting them white on black yields a silhouette control: the moving bodies’ outlines over time, with no distance information at all. Depth is the default because it carries both outline and distance ordering. It stops being informative when a scene is flat—when every object rests on one near-horizontal surface at nearly the same distance from the camera, the depth field is almost constant across the objects and their surroundings, and the control has little geometric contrast to offer. Measured on the rendered controls, the depth separation between a moving body and the surface immediately around it is about twenty grey levels for a ball bouncing on a floor and about a hundred for a ball arcing through open air, but only about fourteen for a rack of balls on a tabletop. For those flat layouts we substitute the silhouette, which restores a maximum-contrast figure-and-ground signal; the price is that distance ordering between the objects is discarded, and the generator has to recover it from the prompt and from ordinary scene priors.

A timing property of the renders. The frame budget and playback rate are fixed—eighty-one frames written at sixteen frames per second—while the simulated interval differs per scene. The clip therefore plays the simulated motion back stretched in time, by factors between roughly $1.8\times$ and $2.3\times$ for the scenes used here. The trajectory shapes and the contact events are the simulation’s; their absolute rate in the finished clip is not, and a downstream measurement of speed or duration must be read against the render clock rather than the simulator’s.

Setting	Value
Backbone	Wan 2.2-VACE-Fun-14B (two-expert MoE), diffusers Wan-VACEPipeline , bf16
Memory	experts CPU-offloaded (two-expert weights exceed an 80 GB accelerator)
Control signal	MuJoCo z-buffer depth (globally normalized); object silhouette for flat scenes
Conditioning	control video + all-white mask + text prompt; no reference image
Resolution	848×480 (flow-shift 3.0)
Frames / rate	81 frames at 16 fps
Sampler / steps	UniPC, 50 steps
Guidance scale	5.0
Conditioning scale	1.0 (swept over $\{0.5, 0.8, 1.0\}$)

Table 1 Deployed generation configuration, from the generation scripts. A small checkpoint provides a pre-flight sanity gate and is not used for final clips.

Relation to structure conditioning generally. Driving synthesis from an explicit geometric control rather than from text alone follows the lineage of ControlNet for images [64] and motion and structure control for video [25, 54]. What differs here is the provenance of the control: it is not estimated from a photograph or drawn by hand but computed from a physics solver, so the structure the generator is asked to follow is physically consistent by construction.

2.3 Pipeline and configuration

Table 1 lists the operating configuration. Generation runs as a staged GPU job: control videos are rendered, a small checkpoint provides a fast sanity gate, the 14B checkpoint generates the clips (data-parallel when more than one GPU is available), and the clips are streamed to storage.

2.4 Design choices

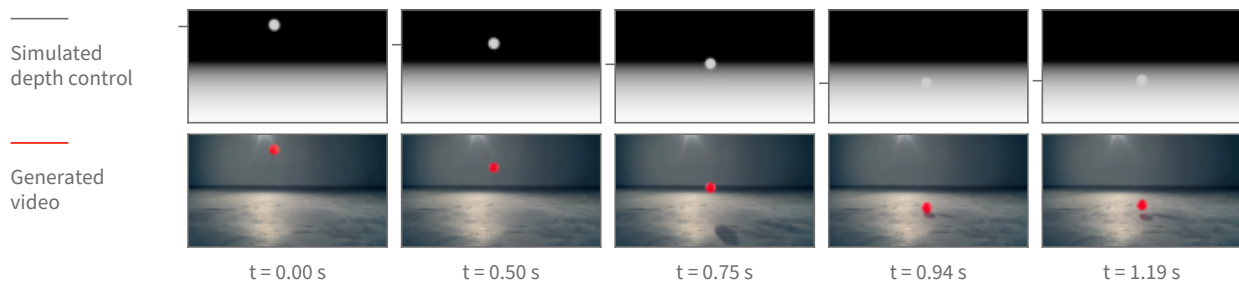
Two choices define the deployed recipe. **Native control rather than source re-synthesis:** a control branch consuming a pre-computed depth video keeps the generator from importing the rendered source’s synthetic look, which re-synthesizing the simulation frames directly does not avoid. **No reference image:** the pipeline accepts an optional appearance reference, but the deployed recipe omits it, because including it degraded single-object trajectories under inspection.

2.5 Scene library and coverage

The two cases above are single rigid bodies because those are the scenes whose trajectory can be checked against a closed-form path. The scene set itself is much wider: 1,314 scenes across 66 kinds. Before any of them was rendered photorealistically, 652 of the underlying simulations were watched and 591 judged to show the intended physical behaviour clearly enough to be worth generating from; that review is of the simulations, not of the generated video. **figure 3** shows twelve of those kinds as real generated frames—cloth draping and crumpling, a flag in wind, a swinging rope and a hanging chain, grains pouring into a heap and draining through a funnel, a ball dropped into a pit of balls, a loaded boat, a cascade down a slope, toppling dominoes and a chaotic two-arm linkage. **figure 4** shows three of them the same way **figure 2** shows a ball: the simulated depth control above, the generated clip below, at matched timestamps.

One limit should be read off these figures explicitly. Everything here comes out of the same rigid-body engine, so the deformables are that engine’s particle-and-link primitives: cloth is a sheet of particles joined by springs, a rope or chain is a short row of linked segments, sand and marbles are many small solid bodies in contact, and floating is a buoyancy force on a solid body in a still fluid volume. None of it is a fluid solve, none of it is

Bouncing ball



Ball thrown across a field

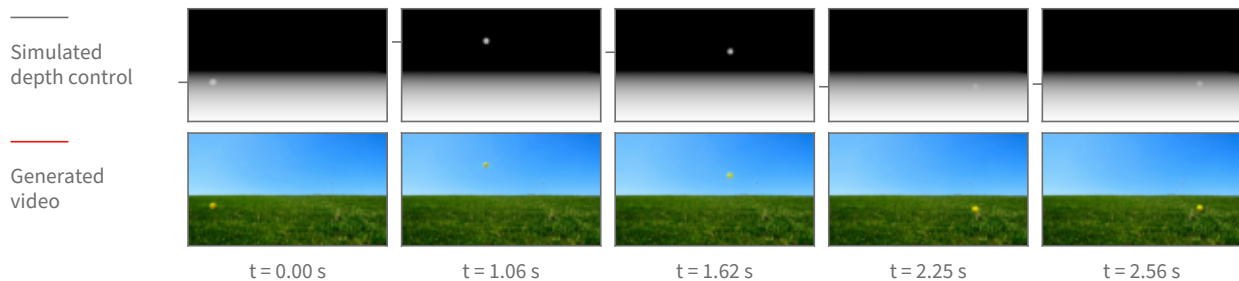


Figure 2 The same mechanism across two scenes with very different requested appearance—an indoor studio and an outdoor field—each shown as its depth control above and the resulting generated clip below. Each scene has its own timestamps, chosen from the turning points of that scene’s tracked vertical path so every column carries motion. The appearance changes completely with the prompt while the motion stays the simulation’s: for the thrown ball the generated and control paths agree to a correlation of 1.0000 horizontally and 0.9998 vertically, with a median disagreement of 0.7 pixels across and 1.1 pixels down, over the 63 frames in which the ball is measurable in both before it leaves the frame.

smoke, and none of it is a finite-element soft body. The scene captions name the primitive rather than the phenomenon for exactly this reason.

2.6 What the generator achieves

Single-object scenes—a bouncing ball, a projectile arc, a swinging pendulum—render cleanly, with the generated object following the simulated trajectory (figure 2). Multi-object interaction, such as a billiards break, remains open: neither the deployed backbone nor the alternates tried against it produce a clean multi-object collision.

These judgments are qualitative. They rest on visual inspection of the output clips and on a color-blob trajectory tracker that checks whether the generated object follows the control’s path. No learned or quantitative video-quality score—FVD, a human-preference model, or a critic score—is applied to this line.

2.7 When the control can be released

The control does not have to be applied for the whole of denoising. A diffusion model builds a video over a sequence of denoising steps that starts from almost pure noise and ends at a clean frame, and the control signal is added in at every one of those steps. That raises a question with both a practical and a scientific side: if the control is switched off partway through, does the motion it has already induced survive to the finished clip? Practically, the steps after the release are free of the control, so the model is free to invent appearance there. Scientifically, the earliest release point that still preserves the motion is the point at which the model has *committed* to that motion.

One frame of the generated video per scene, at the point half the clip’s motion has happened.

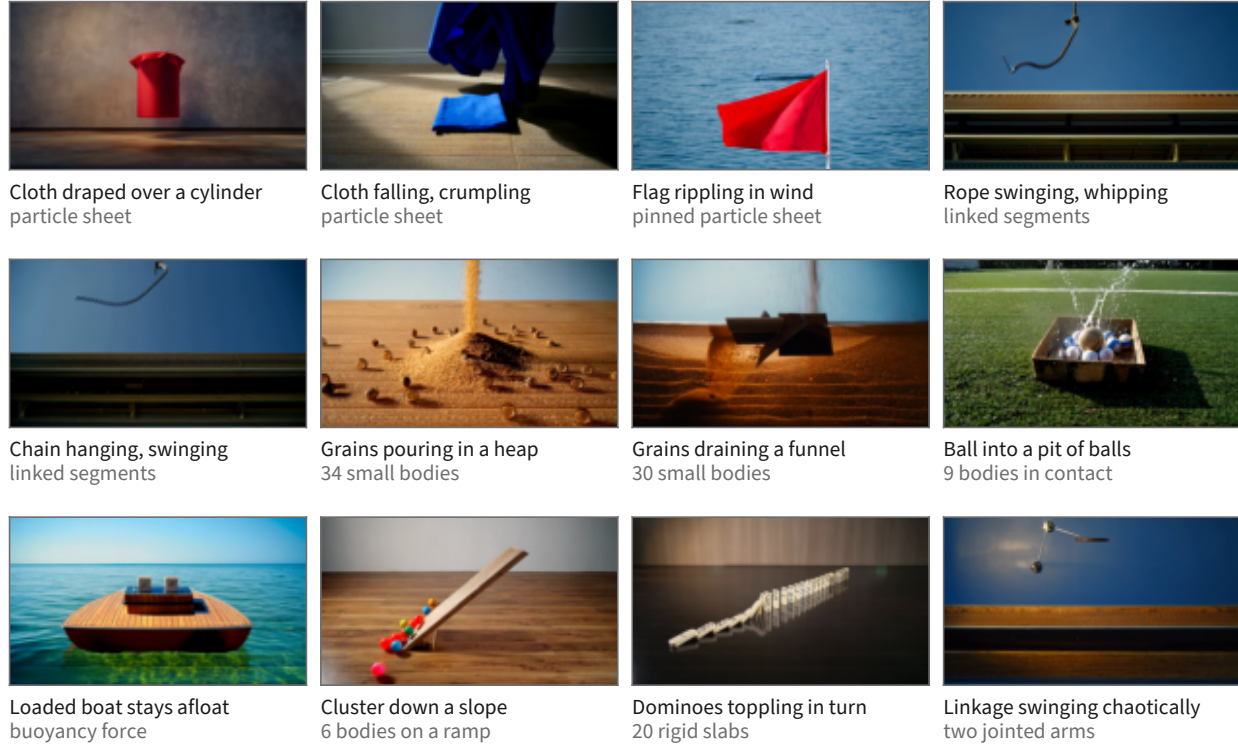


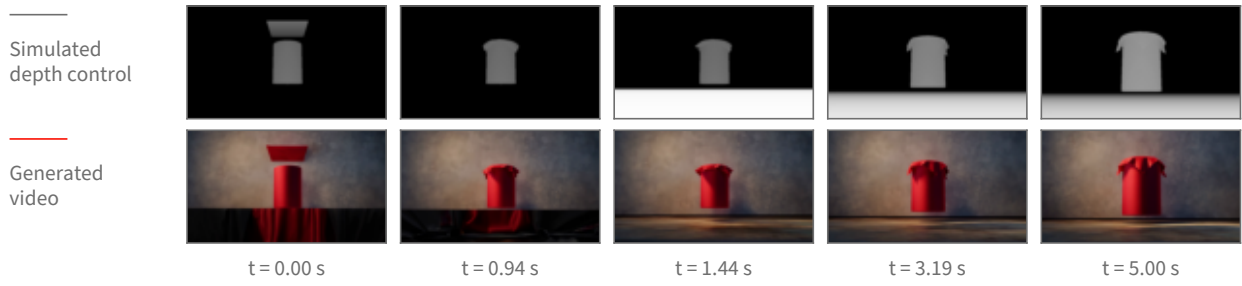
Figure 3 Twelve of the 66 scene kinds, one real generated frame each. The frame shown is computed, not picked: for every clip the frame-to-frame change is accumulated over the whole clip and the tile is the frame by which half of that total change has happened, which lands mid-drape, mid-swing or mid-pour rather than on the first frame. The second caption line names the physics primitive, and every body count in it is the scene’s own parameter. The simulation each clip was generated from passed the pre-generation review described in the text; the generated clips themselves were checked by eye for this figure, which is a weaker guarantee than a systematic review. Two candidate tiles were cut after tracking their control renders showed the phenomenon their label would have claimed does not occur in the clip.

Method. We pick a step, keep the control at full strength for every step before it, and set the control’s contribution to exactly zero from that step onward. Nothing else changes: same simulation, same control video, same prompt, same sampler, same seed. The comparison is always against the same clip generated with the control never released, so the release point is the only difference between the two.

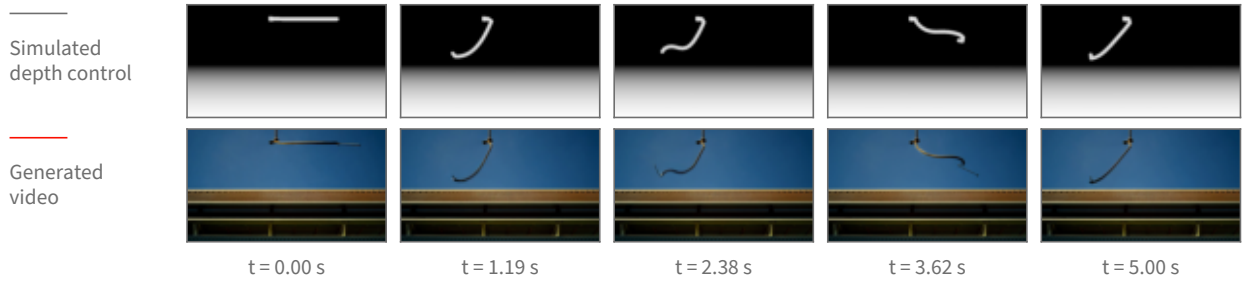
The release point is reported as a noise level, not a step number. A step index is not a property of the model; it is a property of the sampling schedule we happened to choose. The schedule maps each step to a noise level, and that mapping changes when the number of steps or the schedule’s shape changes. Concretely, on the schedule used here, step 8 of 40 sits at a noise level of $\sigma = 0.952$; on a differently-shaped 40-step schedule the same step 8 sits at $\sigma = 0.923$, and the noise level 0.952 has moved to step 5. Reporting “released at step 8” would therefore describe our schedule rather than the model, so every release point below is given as the noise level σ at which the control was switched off, with the step index alongside it only as the label of the run.

What is measured. Both the control video and the generated clip are tracked frame by frame, giving the object’s path in each. *Retention* is how much of the simulated motion the generated clip keeps: the agreement between the generated path and the control’s path, measured separately across the frame (horizontal) and up and down (vertical), because those two directions do not behave the same way. Agreement is a correlation over the frames in which the object is located in both videos, so a value near 1 means the generated object moved the way the simulation did. Two windows are reported, because a single whole-clip number hides

Cloth falling and draping over a cylinder



Rope swinging from a fixed anchor



Grains draining through a funnel

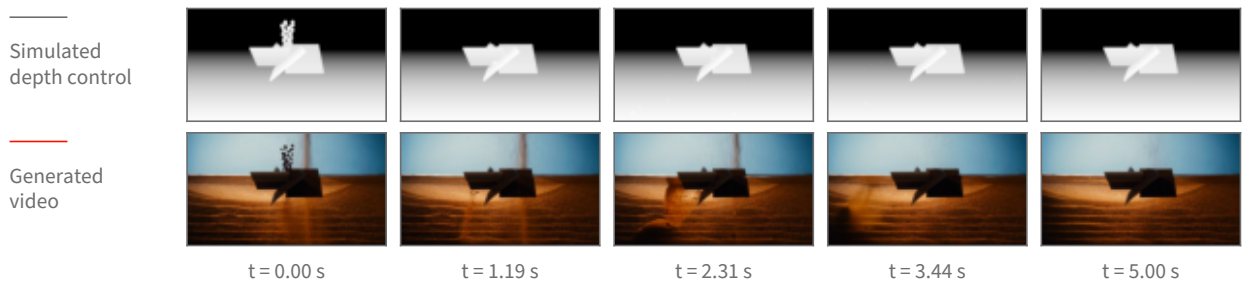


Figure 4 Three deformable and granular scenes shown as [figure 2](#) shows a ball: the simulated depth control above, the generated clip below, at matched timestamps. Columns are placed at equal steps of the clip’s accumulated change so no column falls in a static stretch. These three cases are the ones whose depth control still shows its object in every column—in a depth picture an object that settles onto the floor takes on the same grey as the floor behind it and disappears from the control, so how much of each control frame is distinguishable from its background is measured, and cases whose control goes blank are not shown as pairs.

where a failure lives: the *airborne* window, up to the frame at which the simulated object first reaches the ground, and the *after-contact* window from that frame to the end. The contact frame is not hand-set per scene; it is read off the control’s own vertical path as the first return to the floor after the object’s highest point. Alongside retention we count two failure modes. A *collapse* is a clip whose horizontal agreement falls below 0.7, meaning the sideways motion no longer resembles the simulation’s. A *phantom* is the specific disagreement in which the whole-clip number reads as broken (below 0.7) while the airborne window alone reads as clean (above 0.9) — the signature of a clip whose flight is right and whose ending is wrong. Both thresholds were fixed before scoring.

Result: the flight survives an early release; the ground contact does not. [Figure 5](#) shows the measurement. Two scenes were repeated over 16 seeds each at one release point, $\sigma = 0.952$, in two variants (with and without a reset of the sampler’s internal history at the release), against the never-released baseline: 96 clips in total, 16 per cell. [Figure 5](#) shows that comparison. Releasing the control at $\sigma = 0.952$ leaves the airborne flight of the thrown ball essentially untouched — per-seed horizontal agreement drops by 0.003 relative to never releasing

At one release point (noise level 0.95), 16 seeds per bar

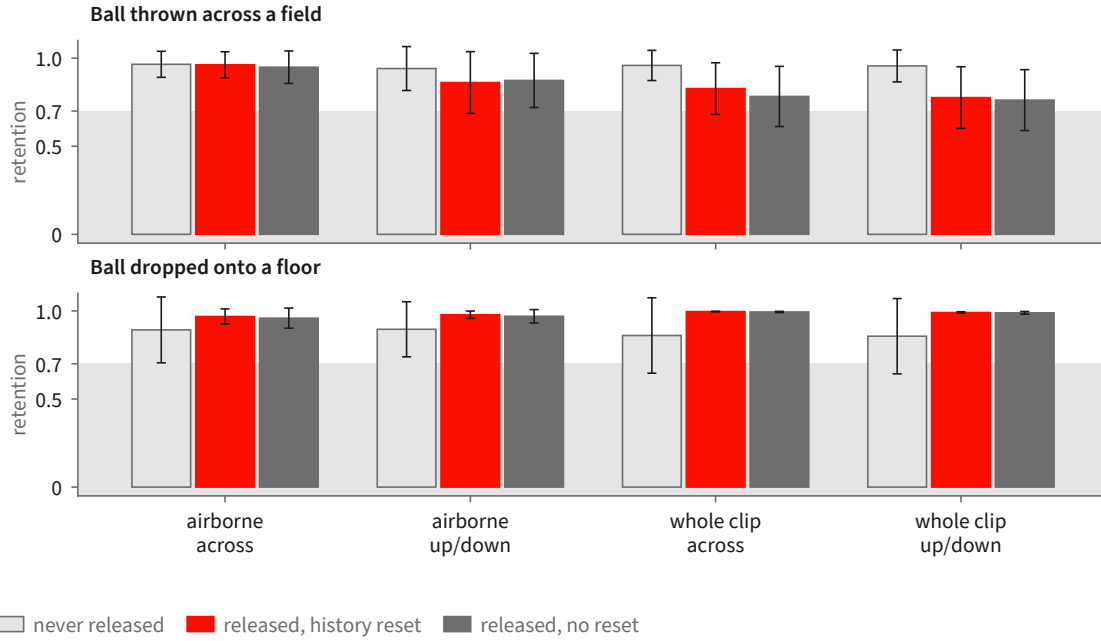


Figure 5 The seeded measurement at the single release point $\sigma = 0.952$, 16 seeds per bar, error bars showing 95% confidence intervals of the mean. Retention is the agreement between the generated object’s tracked path and the simulation’s, so 1.0 is “moved the way the simulation did”. The three bars in each group are the never-released baseline and the two release variants, which differ only in whether the sampler’s internal history is also reset at the release. The airborne window is indistinguishable from never releasing; the whole-clip number is not, and the direction that suffers is the horizontal one. Grey band marks the region below the pre-registered 0.7 break line.

(95% confidence interval 0.001 to 0.004, $n = 16$ paired seeds), which is a real but negligible difference. Over the whole clip the same release costs 0.13 horizontally (95% CI -0.02 to 0.28 , $n = 16$) and 0.18 vertically (95% CI 0.07 to 0.29 , $n = 16$). The gap between those two windows is the finding: what an early release breaks is not the arc but what happens after the ball reaches the ground. Scoring only the frames after the simulated contact, the generated ball’s horizontal motion runs *opposite* to the simulation’s on 9 of 16 seeds when the control is released at $\sigma = 0.952$, and on 0 of 16 when it is never released — the simulation rolls the ball forward and the generator, cut loose, rolls it back. The whole-clip collapse count follows: 3 of 16 seeds at $\sigma = 0.952$ versus 1 of 16 never released, and the phantom count is 2 of 16 versus 0 of 16, so the collapses that do appear are predominantly ending failures on clips whose flight was clean.

Result: a later release is clean, and the vertical direction is the robust one. Sweeping the release point across six settings on a single seed for three scenes gives the shape of the curve (figure 6). For the thrown ball, releasing at $\sigma = 0.952$ gives an after-contact horizontal agreement of -0.63 (the ball travels the wrong way) with the airborne window still at 1.00; releasing at $\sigma = 0.903$ or later restores the after-contact window to 0.98 or above, and it stays there through the latest release point tested, $\sigma = 0.555$. The same sweep on a ball dropped onto a floor never breaks at all — after-contact horizontal agreement is 1.00 at every release point including the earliest, and the vertical agreement, which is the informative direction for a straight-down drop, is lowest at the earliest release (0.78 at $\sigma = 0.952$) and rises to 0.99 or above once the release is at $\sigma = 0.903$ or later. A ball rolling off a ramp holds 0.99 or better at every release point in both directions, but only a third of its frames are measurable at all, so it constrains little. Across all three scenes, vertical motion — the direction gravity supplies — is markedly more robust to an early release than horizontal motion, which the simulation alone specifies.

Retention against the release point

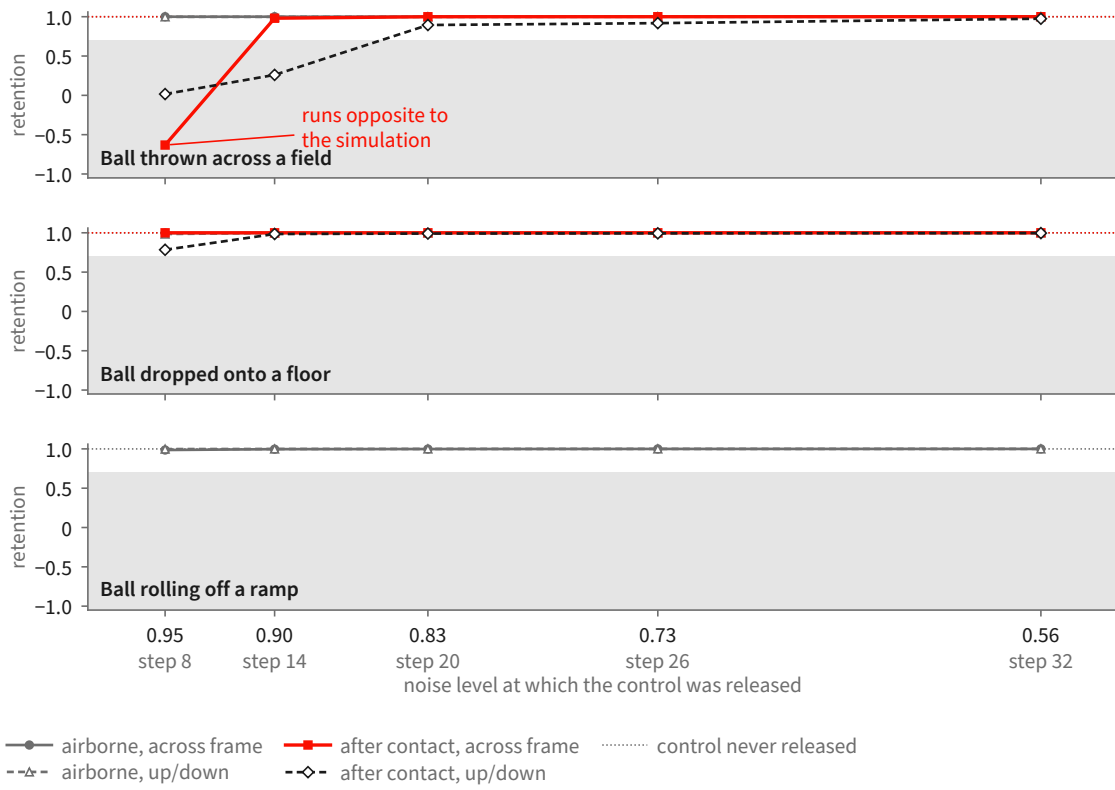


Figure 6 The release-point sweep: one clip per point, three scenes, scored separately over the airborne window and the window after the simulated object first reaches the ground. The horizontal axis is the noise level at which the control was switched off, high noise on the left; the step index of each run is printed underneath as a label only, because the same step index sits at a different noise level under a different schedule. The thrown ball’s after-contact horizontal motion is the one curve that breaks, and it breaks only at the earliest release: it runs *opposite* to the simulation at $\sigma = 0.952$ and is restored from $\sigma = 0.903$ onward. Its airborne flight is unaffected at every release point. Across all three scenes, vertical motion — the direction gravity supplies — is markedly more robust to an early release than horizontal motion, which the simulation alone specifies. Grey band marks the region below the pre-registered 0.7 break line.

A denser sweep, and what it does not settle. A third run sweeps three release points on the thrown ball at three seeds and two guidance strengths, 18 clips, and scores them against the control with a segmentation-based tracker rather than a colour tracker. It reproduces the direction of the effect — airborne horizontal agreement is 1.00 (mean over the 6 clips at $\sigma = 0.952$, 95% CI 0.998 to 1.000), while after-contact horizontal agreement at that same release point averages 0.69 with a confidence interval spanning zero (-0.04 to 1.43 , $n = 6$) and is negative on one clip; at $\sigma = 0.903$ and $\sigma = 0.833$ the after-contact window recovers to 0.99 and 1.00. But it also shows the ceiling on this measurement: at the latest release point the tracker loses the object entirely on 3 of 6 clips, so those cells rest on 3 clips, not 6. On inspection those are clips in which the generated ball is genuinely tiny and low-contrast against the grass, not clips in which the tracker mis-fired, but either way the confidence intervals at the late release points are too wide to place the transition sharply.

Scope. These numbers characterize one conditioning route. The runs condition the backbone by carrying a rendered source video through the denoising process and releasing it partway, rather than through the dedicated control branch of [section 2](#). The question and the noise schedule are the same in both cases, but the release point measured here is a property of the source-carrying route, and its transfer to the control branch is untested.

The scenes are single moving rigid bodies—a ball thrown across a field, a ball dropped onto a floor, a ball rolling off a ramp—at one resolution and frame count. Deformable and granular scenes (section 2.5) are outside the study, because the tracker cannot follow their objects as a single path. Statistical depth is uneven: one release point carries 16 seeds, while the shape of the curve across release points rests on one seed per point for three scenes and three seeds per point for a fourth.

One factor is not isolated. In one of the two variants the release is accompanied by a sampler-history reset, and the study does not separate the two. The variants are statistically indistinguishable on every window and direction measured—the largest per-seed difference is 0.04 horizontally over the whole clip, 95% CI -0.04 to 0.13 , $n = 16$ —which is what one expects if the reset does nothing once the motion has committed, but it does not establish that it never matters.

3 A Human-Annotated Video-Flaw Benchmark

The benchmark is a corpus of 1,500 AI-generated clips in which a person watched each clip against its prompt and wrote down every place the video failed to deliver what the text asked for. Every clip carries at least one such record. The generating model is not recorded in the data, and no clip is attributed to a specific system. The human-written fields are the ground truth throughout this report.

3.1 What a flaw record contains

A flaw is a localized record with six fields. Four are always present: the **violated fragment**, the words of the prompt the video fails, quoted verbatim; a free-text **rationale**; a **severity**; and the annotator’s **confidence**. Two are present when the annotator could supply them: a **time span**, the first and last frame in which the failure is visible, and a **box track**, boxes at the start, middle and end of that span, one per object involved, in normalized coordinates. Table 2 gives each field’s measured coverage across the evaluation core, and figure 7 shows one record in full. Each clip additionally carries an overall severity and a short list of the objects, actions and relations its prompt calls for.

Two coarse tags support grouping and reporting: a *modality* tag saying whether the complaint concerns what is seen, what is heard, or both, and a *type* tag naming the dominant kind of complaint from a fixed ten-item list. Neither is written by the annotator. Both are assigned by three language models classifying each flaw from its quoted fragment and rationale at temperature zero under a fixed seed, resolved by majority; a three-way disagreement resolves to “both” for modality and “other” for type, and a flaw receiving no usable vote falls back to a frozen keyword rule. They are labelled machine-assigned wherever they appear and are never treated as ground truth. Annotation is single-annotator: the corpus carries one annotation per flaw, so no inter-annotator agreement figure is available.

3.2 How the evaluation set is derived

Selection runs in three recorded stages, and the denominator of every later measurement depends on them. Table 3 gives the counts and figure 8 the same derivation as a sequence of steps.

Stage one, clip selection. A clip enters the pool if its video is retrievable and it shows signs of dynamics. The dynamics test is a fixed vocabulary of motion words—falling, bouncing, rolling, colliding, pouring, toppling—applied to the prompt and, separately, to each flaw’s rationale and quoted fragment; a clip is kept when its prompt contains one, or one of its flaws does, or it has a time-windowed flaw against an action-typed object. The test is a keyword rule and selects for prompts and complaints that *talk about* motion, which is a proxy for, not a guarantee of, a flaw whose adjudication needs physical reasoning. This stage keeps 606 of the 1,500 clips, carrying 1,171 flaws.

Stage two, flaw selection. Within those clips, a flaw is a usable detection target only if a critic seeing only pixels could in principle find it. Flaws tagged “seen” or “both” are kept and purely audible complaints dropped, which removes 64 flaws and leaves 1,107 as the full evaluation set.

Stage three, the frozen core. A core of 150 clips with 306 flaws is drawn for routine measurement. The draw is deterministic and its contract recorded: a fixed seed; strata crossing overall clip severity, flaw count per clip

Field	What it holds	Coverage
violated fragment	the words of the prompt the video fails, quoted verbatim; median 8 words	306/306
rationale	free text saying what went wrong instead; median 25 words	306/306
severity	1–5; the distribution is centred at 3 and 4	306/306
confidence	1–5; 239 of 306 are marked 4 or 5	306/306
time span	first and last frame in which the failure is visible	237/306
box track	boxes at the start, middle and end of that span, one per object involved, in normalized coordinates	208/306

Table 2 The six fields of a flaw record and how often each is filled across the 150-clip core. The first four are always present; timing and boxes are not, and a flaw carrying neither is one whose complaint is about the clip as a whole or about something that never appears.

What one human flaw record contains

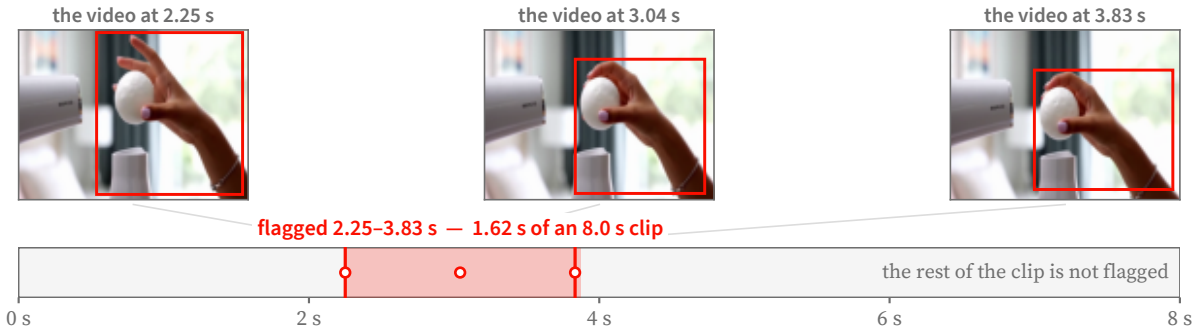
one of the 306 records in the evaluation core

THE PROMPT THE VIDEO WAS ASKED TO SATISFY

A ping pong ball balanced on a hair dryer's air stream, wobbling but never falling. A hand tries to grab it and misses. Audio: Dryer whirl, laughing.

the fragment the video fails to satisfy

WHERE THE FLAW OCCURS



HOW BAD IT IS, AND WHY

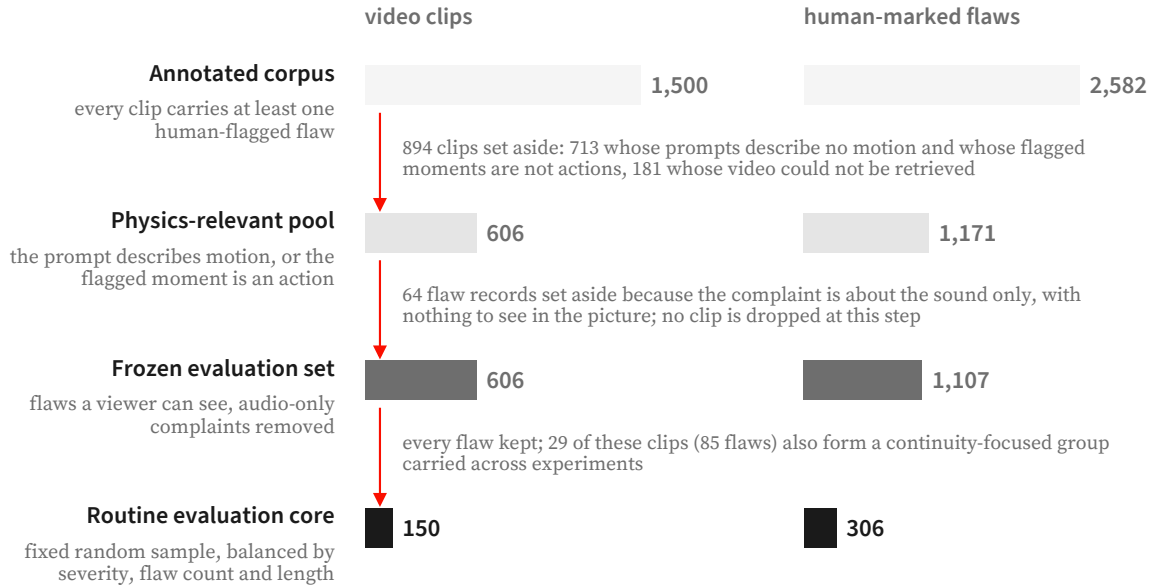
severity ●●●○○ 3 of 5 annotator confidence 5 of 5

the annotator's own reasoning, as written

"The prompt requires "a hand tries to grab it but fails", meaning the object should not be caught. However, it is caught in the actual video, which does not match the requested result."

Figure 7 One human flaw record, shown in full. The annotator quotes the prompt fragment the video fails to satisfy, bounds the interval in which the failure is visible, boxes the objects involved at the start, middle and end of that interval, and writes a severity, a confidence and a rationale in their own words. Frames and boxes are the annotator's; the flagged window is 1.62 s of an 8 s clip. Nothing about the critic's output appears here.

and clip duration; largest-remainder proportional allocation across strata; within each stratum, candidates ordered lexicographically then shuffled once by a generator whose call sequence is fixed; and a replacement rule drawing from the same stratum for any clip that cannot be downloaded or decoded. A continuity-focused stratum of 29 clips and 85 flaws sits inside the core and is exempt from replacement. Every clip is downloaded and decoded before the core is written, and the selection, strata and replacements are stored with the set.



A further 711 annotated clips (1,130 flaws) are held back for training and share no clip with the frozen evaluation set or its routine core. Bar lengths are to scale within each column.

Figure 8 How the evaluation set is derived from the annotated corpus. Clips are removed only at the first step; the second step removes flaw *records* whose complaint is audible but not visible, leaving the clip count unchanged. The routine core is a fixed random sample balanced by severity, flaw count and clip length. A further 711 annotated clips are kept aside for training and share no clip with either frozen set.

Set	Clips	Human flaws
Full corpus (all AI-generated, all annotated)	1,500	2,582
Physics-relevant pool	606	1,171
Full evaluation set (frozen)	606	1,107
Routine evaluation core (frozen)	150	306
continuity stratum within the core	29	85
Disjoint training pool (excludes the frozen sets)	711	—

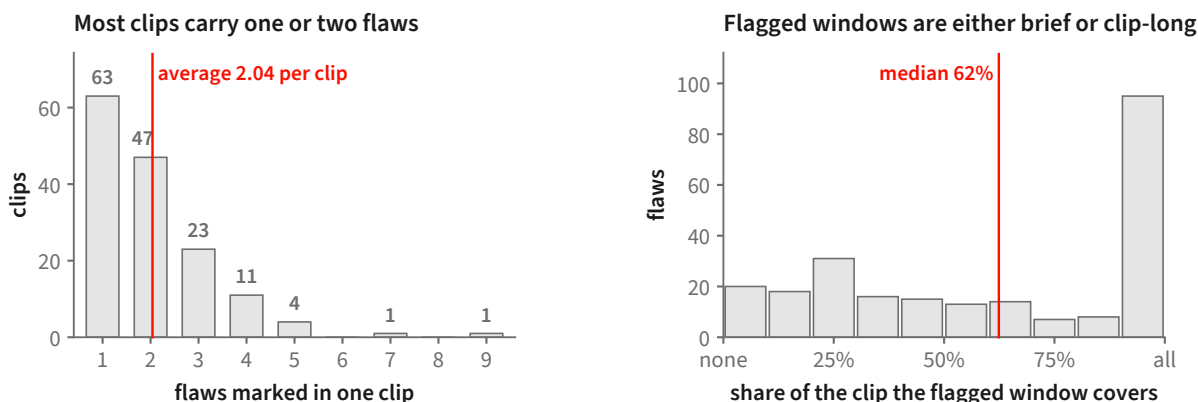
Table 3 Benchmark statistics, from the dataset manifests. The full evaluation set drops a small number of audio-scoped flaws relative to the physics-relevant pool. The training pool excludes every clip in the frozen evaluation sets.

The core is an *optimization set*: system and prompt decisions are made against it, so results on it measure fit to that core, and a generalization claim requires a set that has not influenced system design. A separate pool of 711 annotated clips is held disjoint from both frozen sets for any learning that consumes this data.

3.3 What is in the evaluation core

The core is 150 clips, median 8.0s long (range 5.0–10.3s), mostly at 24fps (118 of 150; the rest at 16, 25, 30 or 32). The prompts are not one-line captions: a median prompt is 42 words and the longest 188, and many specify a shot list, a count, an ordering, an on-screen string and a sound in one paragraph. Sixty-six of the 150 clips carry flaws of more than one kind, so a single number per clip cannot say which of them a detector found.

Its 306 flaws, a median of two per clip, group by machine-assigned type as: actions that do not happen or



150 clips of the routine evaluation core, 306 human-marked flaws. The right-hand panel uses the 237 flaws that carry a start and an end frame; the remaining 69 were marked without one. Of those 237, 57 fit inside a quarter of their clip and 95 run almost its whole length; the median flagged window lasts 5.00 s.

Figure 9 Two properties of the routine evaluation core that shape how a detector must behave. **Left:** most clips are flagged in only one or two places, so a detector that raises many complaints per clip will mostly be raising ones no annotator marked. **Right:** flagged windows are bimodal — many failures are confined to a small part of the clip while a comparable group runs almost its whole length — so both a brief localized check and a whole-clip check are needed.

happen wrongly, 139; wrong counts, 36; wrong or missing objects, 30; garbled on-screen text, 25; wrong spatial arrangement, 19; wrong ordering in time, 17; wrong attributes such as colour or material, 16; camera and style, 10; physical motion, 7; and a residual 7. The largest category is whether the requested action occurred at all, so this is not a physics benchmark in the narrow sense. Counting, text and spatial arrangement together account for 80 flaws, each a question a general-purpose model answers poorly and a dedicated measurement answers well.

Of the 237 flaws carrying a time span, the flagged window covers a median of 62% of the clip, and the distribution is bimodal: 57 are confined to a quarter of the clip or less, and 95 run for at least nine-tenths of it (figure 9). A detector needs both a way to look closely at a moment and a way to judge a whole clip.

3.4 Scoring conventions

Every clip carries at least one flaw. With no clean clips, recall is directly measurable and precision is not: a detector that flags everything scores perfectly on recall alone, so results on this set are reported alongside the rate at which accusations land on a real flaw.

Scoring is over *distinct* flaw spans. Twelve clips record the same failure two or three times under different identifiers—eighteen duplicate entries across the core—and an accusation can be credited to at most one flaw, so identical spans on a clip count as one flaw. The core’s 306 entries are therefore 288 distinct flaws. A run that covers a subset of clips carries its own share of both counts; section 5 states them.

3.5 Relation to existing video benchmarks

The closest annotated resource is Physion-Eval [65], which pairs generated clips with real-world reference processes and provides expert reasoning traces containing temporally localized glitches, a physical-failure category, and an explanation. VideoPhy-2 [6] instead has three annotators rate semantic adherence and physical commonsense, and mark predefined physical rules as violated, followed, or undetermined. Per-dimension suites such as VBench [24] and T2V-CompBench [50] primarily support comparison among generators on fixed prompt collections.

Our corpus targets a complementary, prompt-grounded detector protocol. Each record attaches a failure to the exact words of the generation prompt and permits a time span and box track, so an allegation can

be matched to a particular annotated failure rather than only correlated with a clip-level score. Unlike Physion-Eval, this protocol does not require a matched real-world reference video; unlike VideoPhy-2, it uses open-vocabulary flaw statements rather than a fixed candidate-rule list. These choices make claim-level miss attribution possible in [section 5](#), but they do not improve annotation reliability: the present corpus is single-annotator and contains only flawed clips, whereas VideoPhy-2 provides redundant judgments and majority resolution.

4 PhyReAct: Auditable Physical Verification

4.1 Problem formulation and system overview

A generated clip is paired with the prompt p that requested it. We represent the decoded video by its own container clock,

$$\mathcal{V} = \{(I_t, \tau_t)\}_{t \in \mathcal{T}_\mathcal{V}}, \quad \mathcal{T}_\mathcal{V} = \{0, \dots, T_\mathcal{V} - 1\}, \quad 0 \leq \tau_0 < \dots < \tau_{T_\mathcal{V}-1} \leq D_\mathcal{V}, \quad (2)$$

where I_t is frame t , τ_t is its timestamp, and $D_\mathcal{V}$ is the clip duration. The explicit range on t separates a frame index from time in seconds and also permits nonuniform decode timestamps. An audio track, when present, is addressed on the same clock.

PhyReAct has four roles. A *text-only planner* decomposes the full prompt and writes a verification program before seeing any frame. A *video-aware semantic verifier* reads timestamped frames densely for predicates such as existence and event occurrence. *Specialist instruments* return targeted measurements such as counts, tracks, masks, depths, text, and sound intervals. Finally, a *deterministic aggregator* applies typed predicates and three-valued composition. The planner and semantic verifier may use the same served checkpoint, but they are separate calls with different inputs and responsibilities.

The planner produces exact prompt-grounded claims and a surface plan,

$$\mathcal{C}(p) = \{c_i = (s_i, \phi_i)\}_{i=1}^{N_p}, \quad s_i \sqsubseteq p, \quad (\mathcal{C}, \tilde{\Pi}) = \Pi_\theta(p), \quad (3)$$

where s_i is a verbatim span of p and ϕ_i is the checkable assertion attached to it. The relation $s_i \sqsubseteq p$ is enforced, not inferred after execution. Types belong to the plan’s operations and checks rather than to an unverifiable free-form claim label.

[Figure 10](#) summarizes the resulting dataflow. Planning is *global with respect to the prompt*: the naming pass sees the whole prompt and all later claim groups share that vocabulary. Execution can be *local in time*: a declared interval may gate or scope a later declared action. This is static plan-then-execute orchestration, not global search over alternative plans and not an online ReAct loop. Observations may activate, skip, or localize nodes already in the program; they do not invent a new tool recipe during execution.

4.2 Prompt claims and text-only plan synthesis

Claim extraction preserves both the exact source span s_i and its normalized assertion ϕ_i . A first pass over the whole prompt assigns stable identifiers to every distinct entity and to every separately asserted event occurrence. The claims are then planned in groups of at most four. Each group sees the full prompt, the shared names, and only its assigned claims; its local identifiers are renamed mechanically before all groups are assembled. This construction prevents two groups from silently referring to the same entity by different names or to different occurrences by one name.

The surface language follows code-as-plan visual programming [\[18, 51\]](#), but is deliberately restricted to the operations in [table 4](#). For claim c_i , the planner writes `check(ci, <expr>)` with one operation or a conjunction of operations. At this stage the planner has no video input: the line states what evidence would settle the claim, not whether the claim is true.

Bounded recovery. The assembled surface plan is validated as a whole because its important constraints are cross-references: every check must consume existing actions, every identifier must denote a shared entity or

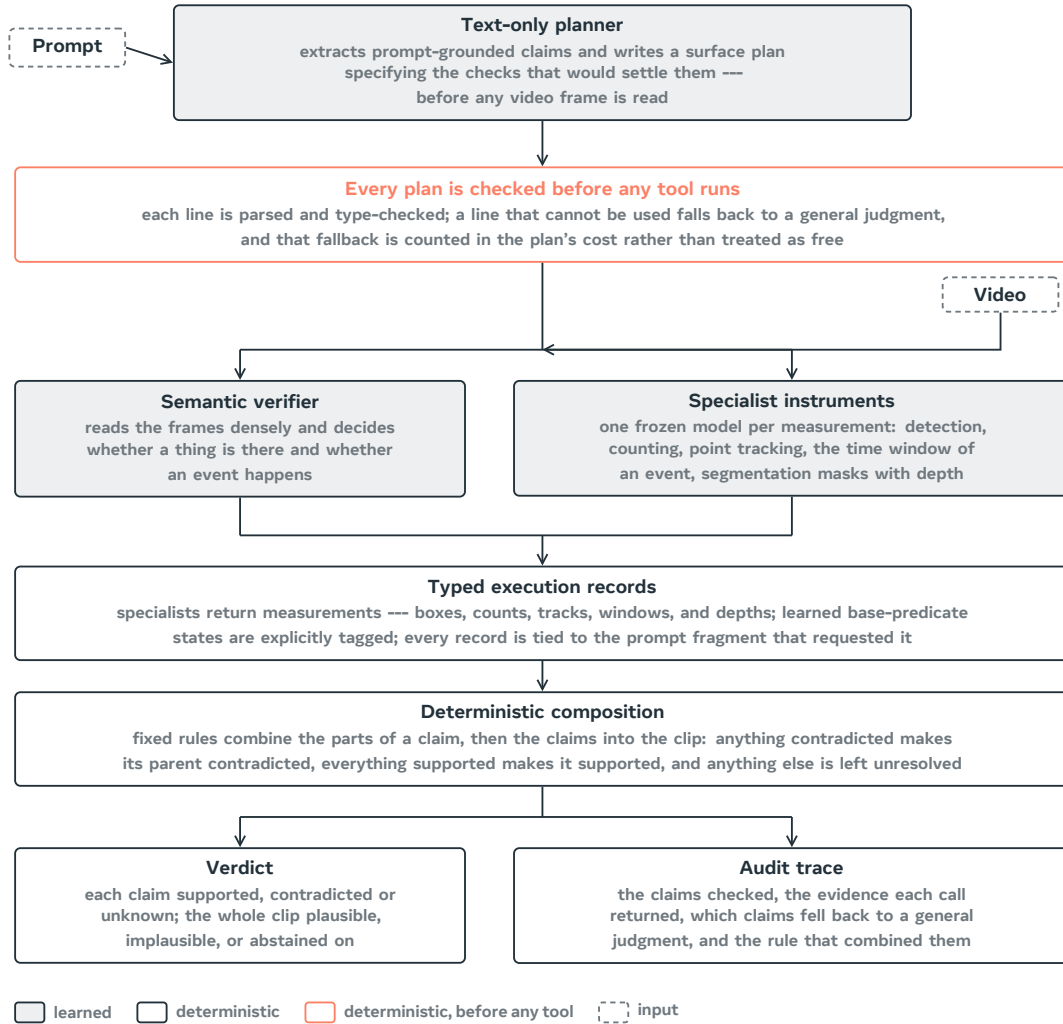


Figure 10 PhyReAct dataflow. The text-only planner receives the prompt and writes claims and a surface plan; the compiler expands and validates it before the video reaches execution. The video-aware verifier emits explicitly learned base-predicate states, whereas specialists emit typed measurements. Only the final claim and clip aggregation is wholly deterministic. Shaded boxes are learned components; outlined boxes are deterministic.

Surface operation	Question posed	Typed check
<code>judge("assertion")</code>	dense-frame semantic verification of the complete assertion	general semantic
<code>exists("object")</code>	whether the object is visible at any decoded time	existence
<code>eq(count("phrase"), N)</code>	whether measured cardinality equals N	count
<code>before(window(A), window(B))</code>	whether event A precedes event B	temporal order
<code>during(window(A), window(B))</code>	whether A is temporally contained in B	temporal containment
<code>rel("A B", relation)</code>	a fixed-vocabulary spatial relation	spatial
<code>traj("phrase", relation, target)</code>	a fixed-vocabulary motion predicate	trajectory
<code>sound("description")</code>	whether the audio track contains the event	audio occurrence
<code>text("STRING", carrier=...)</code>	whether the named surface renders the string	rendered text

Table 4 The planner’s surface vocabulary. `count` and `window` are structural terms. The deterministic compiler expands a surface operation into its required execution closure. Thus `before` adds occurrence gates and two timing actions; `rel` adds co-visibility and mask actions and, for *behind* or *in front of*, depth actions. These dependencies need not be written as extra surface operations, but they must be present in the compiled graph. A `sound` operation reads the audio track; no audio track yields **unknown**, not a negative measurement.

event, and every claim must have a check. A failing group may be asked again with its own parser or type complaints and a restatement of the format, but never with a suggested answer. After a fixed number of attempts, an unusable claim falls back to direct semantic verification. The fallback is recorded and charged as a model call; it is not counted as a specialist measurement. A failed naming pass sends all claims through the same charged fallback, so plan failure never makes a prompt appear cheaper by silently dropping claims.

4.3 Typed plan representation and static validation

The compiler turns the surface program into a typed directed graph

$$G_{\Pi} = (\mathcal{A}, \mathcal{Q}, \mathcal{E}, b), \quad a_j = (o_j, \alpha_j, \omega_j, r_j), \quad b : \mathcal{Q} \rightarrow \{1, \dots, N_p\}. \quad (4)$$

Here $\mathcal{A}$ contains evidence-producing actions, including general semantic passes and specialist calls; $\mathcal{Q}$ contains typed checks; $\mathcal{E}$ points from a prerequisite to its consumer; and $b(q) = i$ binds check q to claim c_i . An action records its operation o_j , arguments α_j , declared scope policy ω_j , and result type r_j .

Before any action runs, the graph must satisfy

$$\text{Valid}(G_{\Pi}) = \text{Parse}(G_{\Pi}) \wedge \text{Refs}(G_{\Pi}) \wedge \text{Typed}(G_{\Pi}) \wedge \text{Acyclic}(G_{\Pi}) \wedge [\forall i, \exists q \in \mathcal{Q} : b(q) = i]. \quad (5)$$

`Refs` checks identifiers and dependency closure; `Typed` checks operator arity, fixed relation vocabularies, and whether every check receives the result types it requires. For example, a directed trajectory must carry a target, and a depth-ordering check must consume both masks and depth. An invalid whole line falls back to the general verifier; an invalid term can be replaced by a recorded generalist sub-check. In both cases the substitution is charged and reported rather than guessed.

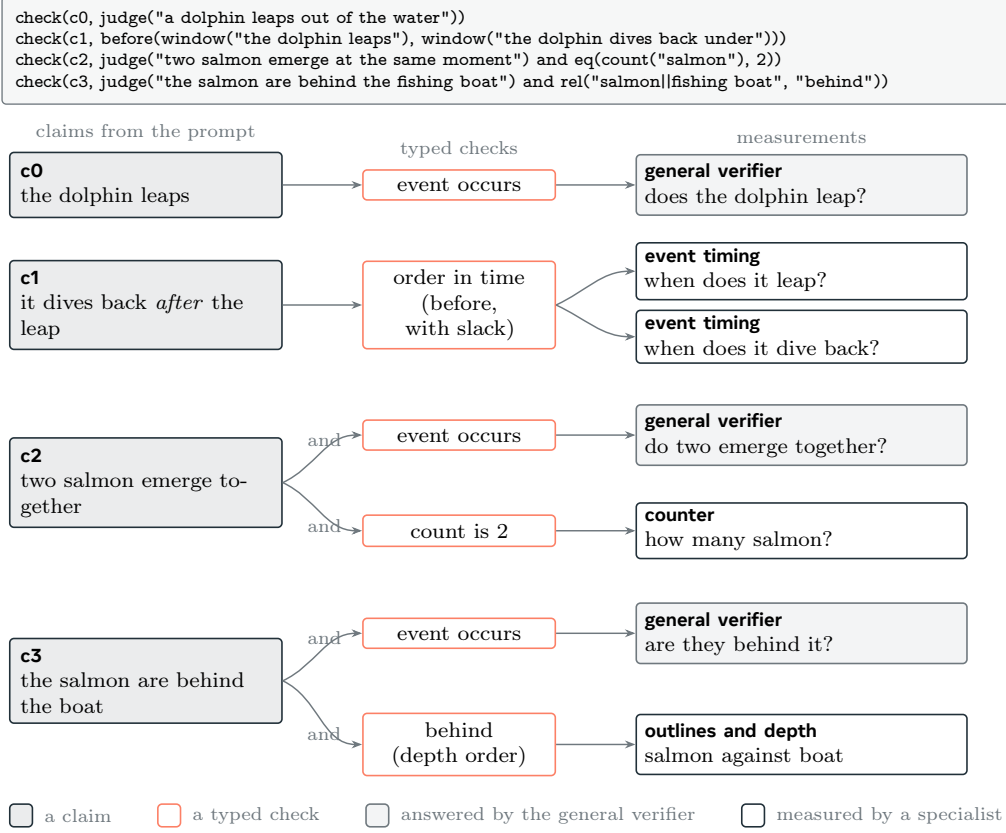


Figure 11 A surface plan after compilation into claims, typed checks, and evidence-producing actions. A numeral adds a count check beside a semantic assertion; an ordering consumes two event windows after occurrence gating; and a depth relation consumes localization, masks, and depth supplied by the compiler’s dependency closure. The graph is illustrative of the planner’s vocabulary rather than an additional learned verdict.

4.4 Timeline-conditioned multi-tool execution

Execution follows a topological order. Each action’s compiled scope function g_j converts prerequisite records into either the full frame index set or a declared local subset, after which the appropriate backend is called:

$$\Omega_j = g_j(\{R_k : (a_k, a_j) \in \mathcal{E}\}, \mathcal{T}_V), \quad R_j = F_{o_j}(\mathcal{V}|_{\Omega_j}, \alpha_j). \quad (6)$$

The scope function is part of the validated program. A usable measured window is padded, given a minimum duration, and converted to the explicit indices whose timestamps fall inside it. A missing, invalid, or overly broad window falls back to the whole clip. This conservative fallback avoids interpreting failure to localize as evidence that the requested event or relation is absent.

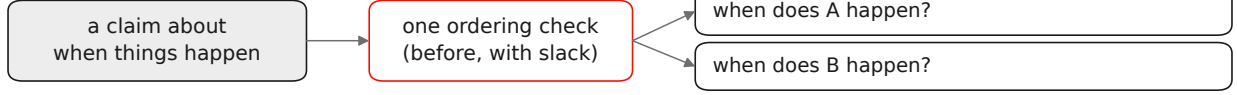
Occurrence gates timing. A returned time window is not evidence that the queried event occurred: a forced localizer may return its best candidate even for an absent event. Visible events are therefore confirmed first by the dense-frame semantic verifier, while audible events are confirmed on the audio track. Only supported occurrences allow their intervals to enter a temporal predicate. For a relation $R \in \{\text{before, during}\}$,

$$z_{R(A,B)} = \begin{cases} C, & C \in \{o_A, o_B\}, \\ U, & U \in \{o_A, o_B\} \text{ or } W_A \text{ or } W_B \text{ is unavailable,} \\ \rho_R(W_A, W_B), & o_A = o_B = S, \end{cases} \quad (7)$$

where o_A, o_B are occurrence states and W_A, W_B are measured windows. Thus timing may compose an order but can never establish occurrence.

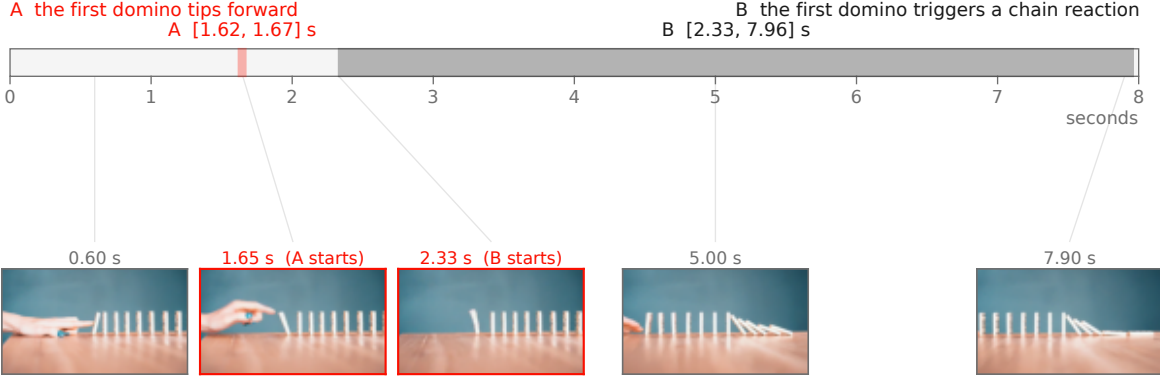
the plan line the planner wrote, before any frame is read

```
check(c1, before(window("the first domino tips forward"),
  window("the first domino triggers a chain reaction")))
```



first, both events must be confirmed to occur by the verifier that reads the frames;
only then do the two windows decide the order

what the two measurements returned, on the clip's own clock



how the order was decided

tolerance = 25% of the shorter window = 0.01 s. A before B iff $A_{end} \leq B_{start} + \text{tolerance}$.
 $1.67 \leq 2.33 + 0.01 \rightarrow \text{yes, so the claim is supported.}$

if neither direction passes, the windows carry no order and the claim remains unresolved.

Figure 12 A real temporal trace. The planner writes the line before reading any frame. The verifier first confirms both events, the timing actions then return $W_A = [1.62, 1.67]$ s and $W_B = [2.33, 7.96]$ s, and [equation \(8\)](#) resolves their order. Every number and displayed frame comes from the recorded run.

Temporal arithmetic. For positive-duration windows $W_X = [s_X, e_X]$, the recorded executor uses a boundary slack equal to one quarter of the shorter window:

$$\delta(A, B) = \frac{1}{4} \min(e_A - s_A, e_B - s_B),$$

$$\rho_{\text{before}}(W_A, W_B) = \begin{cases} S, & e_A \leq s_B + \delta(A, B), \\ C, & e_B \leq s_A + \delta(A, B), \\ U, & \text{otherwise.} \end{cases} \quad (8)$$

The slack absorbs small endpoint uncertainty, so it can tolerate a boundary overlap no larger than δ ; if neither direction passes, the intervals do not resolve an order and the state is U. With the quarter-window rule and positive durations, the first two cases cannot both hold. Containment is directional and uses the same boundary slack: clear containment supports, clear reverse containment contradicts, and boundary-indistinguishable cases remain unknown. These predicates are a tolerance-scaled subset of Allen's interval algebra [1].

Exact-request reuse. Two actions share one execution only when their complete realized signatures match:

$$\chi_V(a_j) = (\text{id}(\mathcal{V}), o_j, \alpha_j, r_j, \Omega_j), \quad a_j \sim a_k \iff \chi_V(a_j) = \chi_V(a_k). \quad (9)$$

The executor calls each equivalence class once, following exact common-subexpression reuse [43, 45]. Because the realized scope Ω_j is part of the signature, whole-clip and localized requests never merge merely because their natural-language queries are similar.

Capability	Running backend			Returned object
Semantic predicate	dense	timestamped	Qwen3-VL	learned base state with localized rationale
Object localization	LLMDet	open-vocabulary	detector [14]	box proposals, not an existence verdict
Object count	CountGD-Box [2, 3] (SAM 2.1 internal)			cardinality of a named set
Kinematics	TAPNext-style point tracking [67] and kinematic predicates			timestamped positions and motion features
Event timing	Qwen3-VL over timestamped frames; optional dedicated grounder [66]			interval, absence, or refusal
Spatial relation	SAM 3 masks [7], monocular depth, and localized reasoning			masks, depth, and a tagged relation state
Rendered text	optical character recognition on localized frames			literal glyph transcription
Audio	FlexSED sound-event detection [19]			sound intervals on the audio track

Table 5 Execution backends and their outputs. Localization proposals seed later measurements but do not prove existence. Specialists provide typed measurements; learned semantic decisions are identified as such. No holistic video-quality model is used to produce the final clip decision.

Execution backends. Table 5 lists the backend that actually runs, rather than a stronger backend that an adapter may only declare. There is no holistic scorer in the verdict path. Three sub-checks remain explicitly learned because they resist full symbolization: subtype identity, dense action-event recall, and depth-ordered spatial interpretation on localized evidence. Their outputs are tagged as learned base-predicate states; they are not presented as deterministic consequences of a raw measurement.

4.5 Evidence records and provenance

Each action is serialized as a claim-bound record

$$R_{j,i} = (\text{id}(\mathcal{V}), p, s_i, o_j, \Omega_j, \sigma_j, \mu_j, h_j, \kappa_j), \quad \sigma_j \in \{\text{measured}, \text{abstained}, \text{errored}\}. \quad (10)$$

The payload μ_j contains either a typed measurement or an explicitly tagged learned base state; h_j contains the native-output and artifact hashes; and κ_j contains calls and accelerator time, with optional wall time, price, and token counts. Shared execution may bind the same record to several claims, but its cost is charged once by execution identifier.

The three statuses are mutually exclusive. A **measured** record carries a payload and no abstention or error; **abstained** carries a reason and no payload; **errored** carries an infrastructure message. A valid negative measurement—for example a measured count of zero or a complete usable track that does not exhibit the requested motion—remains **measured** and may contradict a claim. Failure to track because of blur, occlusion, low quality, or an instrument refusal is **abstained**, not a negative observation. A terminal **errored** record is retried and, if still terminal, produces no semantic verdict; the clip is marked $\perp_{\text{infra}}$ and excluded from the completed evaluation set rather than relabeled **unknown**.

Specialist schemas reject judgment-shaped fields, nonfinite numbers, duplicate keys, and noncanonical payloads. Every record carries the full prompt and the verbatim source span s_i , and heavy artifacts are content-addressed by path and byte hash. The guarantee is intentionally narrow: a specialist cannot emit the final prompt-satisfaction label, and accepted evidence is attributable and replayable. The schema cannot make a learned measurement unbiased or detect a confidently wrong but well-formed payload.

4.6 Three-valued verification and aggregation

For a completed trace, every sub-check resolves in

$$\mathbb{V} = \{S, C, U\},$$

standing for **supported**, **contradicted**, and **unknown**. A typed resolver ψ_q maps measured parent payloads to a sub-check state; it is deterministic for count, timing, track, transcription, and other explicit predicates, and is learned only for the semantic sub-checks named above:

$$z_q = \begin{cases} \psi_q(\{\mu_j : a_j \in \text{Pa}(q)\}), & \sigma_j = \text{measured for all required } a_j, \\ U, & \text{a required action abstained or produced no usable evidence,} \\ \perp_{\text{infra}}, & \text{a required action has a terminal infrastructure error.} \end{cases} \quad (11)$$

Learned semantic checks use an asymmetric gate: they may contradict only when a positively contradicting feature is visible; unresolved visual evidence, unparseable output, or missing modality yields U. This avoids turning failure of an instrument into an accusation about the video.

Negation is applied to the positive sub-check state, never by asking a tool to measure a negative quantity:

$$\neg_3 S = C, \quad \neg_3 C = S, \quad \neg_3 U = U. \quad (12)$$

Let $K_i = \{q \in \mathcal{Q} : b(q) = i\}$ be the validated checks for claim c_i , after applying [equation \(12\)](#) where required. Claim and clip composition are

$$y_i = \begin{cases} C, & \exists q \in K_i : z_q = C, \\ S, & \forall q \in K_i : z_q = S, \\ U, & \text{otherwise,} \end{cases} \quad Y = \begin{cases} \perp_{\text{infra}}, & \text{the trace terminates with an infrastructure error,} \\ \text{IMPLAUSIBLE}, & \exists i : y_i = C, \\ \text{PLAUSIBLE}, & N_p \geq 1 \wedge \forall i : y_i = S, \\ \text{ABSTAIN}, & \text{otherwise.} \end{cases} \quad (13)$$

The order of the cases matters. One contradiction is sufficient for an implausible clip; plausible is emitted only when every planned claim is supported; and a trace with no contradiction but at least one unknown abstains. Plausible therefore summarizes the validated claims that were executed, rather than certifying every possible physical property of the clip.

4.7 Diagnostic interface for future refinement

The useful refinement output is not only the scalar clip state Y , but the localized contradiction packet

$$\mathcal{F}(p, \mathcal{V}) = \{(c_i, \{R_{j,i}\}_j, \Omega_i, B_i) : y_i = C\}, \quad (14)$$

where Ω_i is the union of available temporal support and B_i contains available box tracks or masks. This packet states what failed, which evidence supports that conclusion, and where in the clip the failure occurs.

Closed-loop refinement is not run in this report. The critic and simulation-conditioned generator are evaluated on disjoint data, and no PhyReAct packet has yet been used for preference optimization, denoising guidance, candidate selection, or localized regeneration. [Equation \(14\)](#) defines the interface for that future experiment; it does not claim that the current generator has been optimized by the critic.

5 Evaluation Protocol and Measurement

This section states how the critic is measured and reports what those measurements returned. A dash marks a measurement that has not been run.

Implementation. Qwen3-VL-30B-A3B-Instruct [42], served in BF16 without quantization and tensor-parallel over two GPUs, is used for both the text-only planner and the video-aware semantic verifier. These are separate calls: the planner receives the prompt and claim state but no frames, while the verifier receives densely decoded, timestamped frames and a single declared predicate. The specialist revisions are frozen for the evaluated run and the running backends are those in [table 5](#).

System	Flaws caught of 286	Before collapsing duplicates, of 304	Allegations that landed	Calls per clip
Plan-based critic	197 (68.9%)	197 (64.8%)	40.1%	13.0
Per-claim verification, one claim at a time	179 (62.6%)	179 (58.9%)	40.2%	12.1

Table 6 Paired results on the 149 clips of the evaluation core that both systems completed; the one missing clip is an infrastructure loss. Those clips carry 304 of the core’s 306 flaw entries, which collapse to 286 distinct spans (section 3.1); the second column repeats the same allegations against the uncollapsed flaw list. “Allegations that landed” is the share of each system’s accusations that the matching protocol linked to a human flaw; it is reported because recall alone rewards over-flagging. Calls count model and tool invocations and exclude the matching protocol, which sits outside the critic.

Target and ground truth. The evaluation core of section 3, scored under the conventions of section 3.1. Human labels are the only ground truth.

Matching is outside the critic. To decide whether a critic output corresponds to a human-flagged flaw, a separate protocol uses two judge models (a GPT-family and a Gemini-family model, majority over runs). These judges perform *flaw matching only*; they are not components of the critic, do not produce its verdict, and are not treated as ground truth. Any “majority-of-three” in this project refers to such a model-vote protocol, never to human annotators.

Dense-frame requirement. The critic must observe the video densely. The evaluated components extract frames densely under a logged sampling policy, and any measurement obtained from a sparsely sampled input is excluded. A re-run counts only if it uses the dense path on the unchanged frozen target with BF16 tools at pinned revisions.

The comparison. The critic is compared against per-claim verification: the same claims, verified one at a time by the general verifier, with the question phrased as the critic phrases it, since the phrasing alone changes how readily a verifier answers. Because a detector call, a tracker call and a full vision-language judgement do not cost the same, cost is reported as model and tool calls per clip. Uncertainty is estimated by resampling whole clips, because flaws within a clip are not independent.

Planner validity and measurement coverage. Before end-to-end evaluation, the prompt-only planning stage was inspected without video or tool access. On 233 claims from 22 annotated clips, every claim produced a valid, compilable plan at one to two planner calls per clip. Twenty-seven claims (11.6%) contained at least one specialist action that survived validation, compared with 2.3% for an earlier tabular plan format. On deliberately measurement-dense diagnostics the corresponding rates were 19/30 for counting, spatial, and ordering claims, and 10/10 for synthetic counting prompts. These diagnostics test routing and syntax; they are not estimates of verdict accuracy or coverage on the evaluation core. The present surface vocabulary also lacks an “on top of” spatial predicate and rejects negated quantitative measurements; such terms become recorded, charged generalist fallbacks.

5.1 Results

Table 6 reports the paired comparison completed to date: the plan-based critic against per-claim verification, on the 149 clips of the evaluation core both systems have scored. Both receive the same claim texts and the same matching protocol, and the per-claim baseline is wording-controlled—asked its questions in the form the plan-based critic uses, because the phrasing alone changes how readily a verifier answers. Scoring follows section 3.1.

The plan-based critic catches **18 more flaws**, 6.3 points of recall, for about one extra call per clip. Resampling whole clips with a fixed seed puts that difference at 95% confidence in [+8, +28], so it is a real difference on this set rather than sampling noise.

Judge	All 200 clips				Hard 55	
	exact	± 1	PLCC	SRCC	PLCC	SRCC
VideoScore2 [21]	33.0%	83.0%	0.353	0.327	0.239	0.151
This work	31.0%	76.0%	0.464	0.465	0.480	0.460

Table 7 Prompt match: agreement with the human semantic-adherence rating on 200 pre-registered VideoPhy-2 test clips, scored with VideoScore2’s own evaluation script. VideoPhy-2 also carries a physical-commonsense rating; the critic emits no score on that scale, so that dimension is outside this comparison.

Two qualifications bound what the number means. It is a trade of accuracy against cost, not a dominance result: the plan-based path spends about 7% more calls per clip, and no run holds the two systems to an equal call budget. And this set is the set the system was developed against, so the figure describes fit to it rather than generalization.

Where the misses come from. Of the 89 flaws the plan-based critic missed, two were never decomposed—no claim in the plan covered them. Eight were flagged but not linked to the human flaw by the matching protocol. The remaining seventy-nine were checked and cleared: a claim covered the flaw and the critic judged the video fine, and seventy-six of those were decided by a yes-or-no judgement with no measurement attached rather than by a specialist returning a wrong number.

The bottleneck is therefore measurement coverage rather than decomposition or specialist accuracy. Of 1,917 executed checks, 1,414 ask whether an event occurs and 233 whether something exists, against 22 spatial relations, 19 trajectories, 18 orderings, 18 text reads and 16 counts: 14.1% are measurements. The claims this data produces rarely ask the plan to measure anything, which makes claim typing, not tool accuracy, the next lever.

Cost. Per-clip wall time over the 149 clips has a median of 102 s and a mean of 373 s; the ninetieth percentile is 1,087 s and the maximum 6,119 s. The mean sits far above the median because a minority of clips invoke dense per-frame specialists, and those clips set the wall-clock rather than the typical one.

What this does not cover. The paired set is 149 of the 150 benchmark clips. The cost-matched comparison in both directions and per-clip GPU-second accounting remain unrun, as does every flaw-level external comparison in [table 8](#); the rating-level comparison of [section 5.2](#) is separate and has been run.

5.2 Agreement with human ratings, against a purpose-trained judge

Flaw recall is not commensurate with what most published evaluators emit, which is a clip-level scalar rather than an allegation that can be matched to a localized human flaw. There is, however, common ground: both kinds of system can be asked to rate a clip on the scale a human used. VideoScore2 [21] is the natural opponent there—a judge trained for exactly this task, with a published evaluation script.

The comparison follows that script rather than a protocol of our own. On a sample of VideoPhy-2 [6] test clips, each judge emits an integer 1–5 per dimension; agreement with the human rating is scored as exact accuracy, accuracy within one point, and the linear and rank correlations (PLCC and SRCC), with unparsed predictions excluded pairwise. The sample is 200 clips drawn uniformly with a fixed seed, and the sample, the score definition and the metrics were registered before any result existed; nothing in the scoring function is fitted to these clips. Of the 200, 55 carry the benchmark’s *hard* flag.

[Table 7](#) reports both judges, and the two families of metric disagree: neither judge leads on both. VideoScore2 is better calibrated to the human’s absolute point on the scale, winning exact agreement (33.0% against 31.0%) and agreement within one point (83.0% against 76.0%). The critic correlates better with the human ordering. Bootstrapping the paired per-clip difference over the same 200 clips, the rank-correlation advantage is +0.138 with a 95% interval of [+0.005, +0.277], which excludes zero only narrowly; the linear-correlation

Evaluator	Output it produces	Caught	Recall	Calls
This work, plan-based	localized allegation	197	68.9%	13.0
Per-claim verification	localized allegation	179	62.6%	12.1
Question-decomposition scoring [9, 23]	per-question answers	—	—	—
Alignment scoring [22, 29]	clip-level scalar	—	—	—
Learned quality scoring [20, 21]	clip-level scalar	—	—	—
Modular video question answering [11, 38]	per-question answers	—	—	—

Table 8 The intended external comparison, stated before it is run. The first two rows are the measured systems of table 6, on its 149 clips and 286 flaws. Dashes are unrun. A clip-level scalar has no allegation to match against a localized human flaw, so scoring it requires a stated rule for turning a score into a decision; that rule will be fixed in advance and reported with the result.

advantage is +0.110 with an interval of $[-0.018, +0.242]$, which does *not* exclude zero. On the full sample, then, the ordering advantage is real but slight, and the linear one is not established.

The hard slice is where the two separate. On those 55 clips the critic’s advantage is +0.242 linear ($[+0.018, +0.511]$) and +0.309 rank ($[+0.050, +0.583]$), both excluding zero, as VideoScore2’s rank correlation falls to 0.151 while the critic’s holds at 0.460. That is the pattern one would expect if evidence assembled per claim degrades more gracefully on difficult clips than a single learned scalar, though 55 clips is a small sample and the intervals are correspondingly wide.

Why only one of the two scales is reported. VideoPhy-2 rates each clip on a second scale, physical commonsense, and the registered plan mapped the critic’s output into that column as well. That analysis is not reported as a result, because it does not measure what the column is for. The critic’s score is the fraction of a clip’s claims that its checks contradicted, converted to a 1–5 number; rating how physically plausible a video looks is a different output from deciding whether a stated claim is contradicted, and the critic emits no score on the former scale. Comparing the one against the other measures the mismatch, not the capability. Producing such a rating would need a head that consumes the kinematic and trajectory checks directly, which does not exist here. This analysis therefore provides no evidence about physical-commonsense rating, and none about physics-flaw detection either; the flaw-level evidence for that is table 6, on human-localized flaws.

Two further limits. This is one fixed sample of 200 clips, so it bounds agreement on that sample rather than across the benchmark. And the two judges are not cost-matched: the critic runs a decomposition and a set of tool calls per clip, VideoScore2 a single forward pass, so nothing here speaks to efficiency.

5.3 The intended flaw-level comparison

The rating comparison above does not settle the flaw-level question, because a clip-level scalar cannot be matched to a localized human flaw. Table 8 fixes that comparison, so that the protocol is stated before the numbers exist. Each will be scored on the same frozen set, against the same human labels, with the same matching protocol, and each will be given the same claim decomposition where it accepts one.

The set was inspected and used during optimization, the wording-controlled per-claim baseline was introduced after treatment outputs had been observed, and no retroactive held-out split exists within these 150 clips; a claim of generalization therefore requires evaluation on a new frozen set.

6 Related Work

6.1 Physics-aware video generation and refinement

Controllable diffusion separates appearance synthesis from structural guidance. ControlNet introduced trainable branches for conditions such as depth and segmentation [64], VideoComposer extended compositional control to space and time [54], and Wan and VACE provide a scalable video backbone and a unified interface for reference, mask, and structural conditions [25, 53]. These methods determine how a condition enters a generator, but not whether that condition or the rendered pixels obey physics.

A complementary line derives guidance from physical reasoning or simulation. MotionCraft warps diffusion latents with simulated optical flow, while PhysGen estimates scene properties, performs rigid-body simulation, and refines the result with a video prior [31, 39]. DiffPhy injects language-model-inferred physical context through supervised fine-tuning [63]; PhyT2V iteratively rewrites prompts from physical-rule and mismatch reasoning [58]; and VLIPP turns vision-language reasoning into a coarse motion plan [60]. PhyCo learns continuous controls for physical properties from simulation data [41]. More tightly coupled systems place a simulator inside generation: PSIVG reconstructs a generated template, simulates object trajectories, and feeds them back at test time [13], while MoReGen coordinates language agents, simulation, and rendering for code-based synthesis [4].

Evaluation signals have also entered the generation loop. Binary VLM feedback has been used for preference optimization of object interactions [15]; Video-T1 searches candidate denoising trajectories with test-time verifiers [30]; and VideoRepair converts detected semantic misalignment into localized regeneration [27]. Our generation testbed is deliberately narrower: known MuJoCo state is rendered into metric depth or silhouettes for a frozen Wan 2.2-VACE model. This avoids inverse reconstruction and generator training, but a physically coherent control signal does not certify that the synthesized pixels preserve its physics. The present work therefore evaluates the generator and critic separately; using PhyReAct evidence to guide generation remains future work.

6.2 Generated-video evaluation and physical benchmarks

General-purpose suites measure complementary aspects of generated video. VBench, T2V-CompBench, FETV, and EvalCrafter aggregate perceptual quality, temporal consistency, and prompt-alignment criteria [24, 32, 33, 50], while VideoScore and VideoScore2 learn to predict fine-grained human judgments [20, 21]. Prompt decomposition is also established: TIFA and the Davidsonian Scene Graph turn text into questions or propositions [9, 23], and VQAScore evaluates a text condition with an image-to-text model [29]. These approaches are valuable for ranking systems, but their final scores do not by themselves expose a measurement trace for a specific physical allegation.

Physics-specific evaluation has rapidly broadened. VideoPhy and VideoPhy-2 assess semantic adherence and physical commonsense, with VideoPhy-2 additionally labeling candidate physical rules as followed, violated, or undetermined [5, 6]. PhyGenBench spans common physical phenomena [36], Physics-IQ tests whether generative models encode physical principles [40], and PhyCoBench compares observed motion with optical-flow-guided future-frame prediction [8]. PhyWorldBench extends coverage to fundamental, composite, and deliberately anti-physical prompts, with prompt-specific evaluation standards [17]. These benchmarks provide essential generator-level comparisons and increasingly structured criteria. Our question is complementary: given an arbitrary prompt and clip, can an evaluator expose which physical obligation failed, what instrument measured it, when the supporting evidence occurred, and why insufficient evidence produced an abstention rather than a pass?

6.3 Agentic tool use and neuro-symbolic video reasoning

ReAct established the reasoning–action–observation loop, in which tool results can change the next reasoning step [62]. Related work explores how such behavior is acquired and organized. Toolformer learns when and how to insert API calls through self-supervision, while GPT4Tools transfers multimodal tool use to open language models through self-instruction [44, 59]. HuggingGPT uses an LLM controller to plan and route subtasks across expert models [47], while ControlLLM and ToolChain* search tool-call graphs or action trees

[34, 68]. ReWOO and LLMCompiler instead plan before execution and make tool dependencies explicit [26, 57]. Feedback-based agents close another loop: Reflexion stores linguistic feedback in episodic memory across trials, whereas CRITIC invokes external tools to validate and iteratively revise an answer [16, 48].

Multimodal systems specialize these ideas to perception. Visual programs such as VisProg, ViperGPT, and CodeVQA compose vision modules through generated programs [18, 49, 51], while ProViQ, MoReVQA, the VideoAgent systems, and VideoTree adapt modular or selective observation to video [11, 12, 38, 55, 56]. Recent video agents make observation itself adaptive: CAViAR invokes modules and uses a critic to select reasoning traces, and LensWalk controls temporal scope and sampling density [28, 37]. Their primary setting is video question answering rather than physical verification of generated content.

Three systems are particularly close. VideoGen-Eval combines LLM prompt structuring, MLLM judgments, and temporal patch tools for multidimensional generated-video evaluation [61]. NeuS-V translates prompts into temporal-logic specifications, maps frame-level proposition confidence to a video automaton, and uses probabilistic model checking for alignment [46]. NeuS-E then identifies a weak proposition and failure frame and edits or regenerates the affected suffix [10]. Thus, agentic evaluation, prompt-to-temporal specification, and verification-guided repair all predate this work.

The name **PhyReAct** deliberately echoes ReAct, but its control flow follows the plan–execute branch rather than a free-form thought–action–observation loop. Before viewing the video, it compiles and validates typed physical claims and their dependencies; during execution, observations may gate or scope already-declared measurements but do not trigger unconstrained replanning or search over alternative tool recipes. Count, tracking, segmentation, depth, OCR, audio, and reasoning specialists must return evidence rather than prompt-level verdicts, and fixed composition rules preserve **supported**, **contradicted**, and **unknown** with claim-level provenance. The present system neither trains a tool-use policy nor maintains reflective memory across clips. Unlike NeuS-V, its interface covers measurements that are not naturally reducible to frame-level proposition confidence, but it also provides no formal guarantee when its learned perception is wrong. Unlike NeuS-E, Reflexion, or CRITIC, it does not yet turn its diagnostics into autonomous online correction.

6.4 Diagnostic supervision for generated-video failures

Human supervision for generated video ranges from holistic ratings and preferences to dimension-level explanations. VideoGen-Eval evaluates human agreement over more than 12,000 videos generated from 700 structured prompts [61]. Physion-Eval is closer to flaw diagnosis: its 12,718 generated videos and 10,990 expert reasoning traces cover 22 physical categories, temporally localize failures, and use corresponding real-world videos as references [65].

Our 1,500-clip corpus is complementary rather than a replacement. Its 2,582 human-written records bind each flaw to the violated prompt fragment and a rationale, severity, and confidence, with an optional time span and box track. This schema supports open-vocabulary allegation-to-flaw matching for a prompt-conditioned critic without a matched real-world reference. The resulting benchmark is more directly aligned with **PhyReAct**’s output contract, but its all-flawed, single-annotator construction supports recall measurement only and does not provide the precision or agreement statistics of a balanced, redundantly annotated benchmark.

7 Limitations

Generation. The control signal comes from a simulator, so scene diversity is bounded by what is simulated, and the mapping from simulation to prompt appearance is not itself verified for physical consistency.

Benchmark. The corpus is recall-only (no clean clips), so precision is not directly measurable; flaws are single-annotator with no inter-annotator agreement; the scope/category tags are machine-derived; and the source generator of the clips is unrecorded. Repeatedly designing against one frozen set risks overfitting to it even without training on it.

Critic. The spatial relation predicate and negative-event verification are limited; the typed-plan layer does not yet dispatch the full specialist registry; binding remains a central failure mode, since the compiler enforces

symbolic identifier consistency but cannot guarantee that heterogeneous tools localized the same physical instance; and the reasoner-mediated sub-checks reduce but do not remove semantic opacity.

8 Conclusion

Physical faithfulness in generated video requires steering, measurement, and ground truth together. This report contributes one piece for each: a generator that imports geometry and motion from a simulation and renders them through a video-diffusion control branch; a human-annotated benchmark that localizes real flaws in space, time, and prompt reference; and an auditable multi-tool critic that compiles a prompt into typed checks, combines explicitly identified learned base predicates with specialist measurements, and rolls them up deterministically into a three-valued verdict and localized trace. Against human labels, the compiled-plan critic recovers more localized flaws than verifying each claim on its own, at a slightly higher call count; an equal-cost comparison and a held-out set remain to be run. Closing the loop that lets the critic refine the generator is the next milestone.

A Appendix

To be expanded: the full generation configuration and control-render details; the benchmark schema and per-stratum statistics; the critic’s plan schema, predicate type rules, and temporal-relation truth tables; the specialist license and revision manifest; and the pre-registered evaluation manifest.

References

- [1] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434.
- [2] Niki Amini-Naieni and Andrew Zisserman. Open-world object counting in videos. In *AAAI*, 2026. arXiv:2506.15368. The CountVid model; the count arm.
- [3] Niki Amini-Naieni, Tengda Han, and Andrew Zisserman. Countgd: Multi-modal open-world counting. In *NeurIPS*, 2024. arXiv:2407.04619.
- [4] Xiangyu Bai, He Liang, Bishoy Galoaa, Utsav Nandi, Shayda Moezzi, Yuhang He, and Sarah Ostadabbas. Moregen: Multi-agent motion-reasoning engine for code-based text-to-video synthesis. In *CVPR*, pages 7632–7642, 2026. arXiv:2512.04221.
- [5] Hritik Bansal, Zongyu Lin, Tianyi Xie, Zeshun Zong, Michal Yarom, Yonatan Bitton, Chenfanfu Jiang, Yizhou Sun, Kai-Wei Chang, and Aditya Grover. Videophy: Evaluating physical commonsense for video generation. *arXiv preprint arXiv:2406.03520*, 2024.
- [6] Hritik Bansal, Clark Peng, Yonatan Bitton, Roi Goldenberg, Aditya Grover, and Kai-Wei Chang. Videophy-2: A challenging action-centric physical commonsense evaluation in video generation. *arXiv preprint arXiv:2503.06800*, 2025.
- [7] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Dídac Surís, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025. Open-vocabulary concept segmentation/tracking; used by the relation depth arm.
- [8] Yongfan Chen, Xiuwen Zhu, and Tianyu Li. A physical coherence benchmark for evaluating video generation models via optical flow-guided frame prediction. *arXiv preprint arXiv:2502.05503*, 2025.
- [9] Jaemin Cho, Yushi Hu, Roopal Garg, Peter Anderson, Ranjay Krishna, Jason Baldridge, Mohit Bansal, Jordi Pont-Tuset, and Su Wang. Davidsonian scene graph: Improving reliability in fine-grained evaluation for text-to-image generation. In *ICLR*, 2024. arXiv:2310.18235.
- [10] Minkyu Choi, S P Sharan, Harsh Goel, Sahil Shah, and Sandeep Chinchali. We’ll fix it in post: Improving text-to-video generation with neuro-symbolic feedback. *arXiv preprint arXiv:2504.17180*, 2025.
- [11] Rohan Choudhury, Koichiro Niinuma, Kris M. Kitani, and László A. Jeni. Video question answering with procedural programs. In *ECCV*, pages 315–332, 2024. arXiv:2312.00937; the video analogue of ViperGPT.
- [12] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. Videoagent: A memory-augmented multimodal agent for video understanding. In *ECCV*, 2024. arXiv:2403.11481.

- [13] Lin Geng Foo, Mark He Huang, Alexandros Lattas, Stylianos Moschoglou, Thabo Beeler, and Christian Theobalt. Physical simulator in-the-loop video generation. In *CVPR*, pages 4301–4311, 2026. arXiv:2603.06408.
- [14] Shenghao Fu, Qize Yang, Qijie Mo, Junkai Yan, Xihan Wei, Jingke Meng, Xiaohua Xie, and Wei-Shi Zheng. LlmDET: Learning strong open-vocabulary object detectors under the supervision of large language models. In *CVPR*, 2025. arXiv:2501.18954.
- [15] Hiroki Furuta, Heiga Zen, Dale Schuurmans, Aleksandra Faust, Yutaka Matsuo, Percy Liang, and Sherry Yang. Improving dynamic object interactions in text-to-video generation with ai feedback. *arXiv preprint arXiv:2412.02617*, 2024.
- [16] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. Critic: Large language models can self-correct with tool-interactive critiquing. In *ICLR*, 2024. arXiv:2305.11738.
- [17] Jing Gu, Xian Liu, Yu Zeng, Ashwin Nagarajan, Fangrui Zhu, Daniel Hong, Yue Fan, Qianqi Yan, Kaiwen Zhou, Ming-Yu Liu, and Xin Eric Wang. Phyworldbench: A comprehensive evaluation of physical realism in text-to-video models. In *ICLR*, 2026. Oral; arXiv:2507.13428.
- [18] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *CVPR*, 2023. arXiv:2211.11559.
- [19] Jiarui Hai, Helin Wang, Weizhe Guo, and Mounya Elhilali. Flexsed: Towards open-vocabulary sound event detection. *arXiv preprint arXiv:2509.18606*, 2025. Text-queryable sound-event detection with timestamps; the audio arm.
- [20] Xuan He, Dongfu Jiang, Ge Zhang, Max Ku, Achint Soni, et al. Videoscore: Building automatic metrics to simulate fine-grained human feedback for video generation. In *EMNLP*, 2024. arXiv:2406.15252.
- [21] Xuan He, Dongfu Jiang, Ping Nie, Minghao Liu, Zhengxuan Jiang, et al. Videoscore2: Think before you score in generative video evaluation. *arXiv preprint arXiv:2509.22799*, 2025.
- [22] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. CLIPScore: A reference-free evaluation metric for image captioning. In *EMNLP*, 2021. arXiv:2104.08718.
- [23] Yushi Hu, Benlin Liu, Jungo Kasai, Yizhong Wang, Mari Ostendorf, Ranjay Krishna, and Noah A. Smith. TIFA: Accurate and interpretable text-to-image faithfulness evaluation with question answering. In *ICCV*, 2023. arXiv:2303.11897.
- [24] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *CVPR*, 2024. arXiv:2311.17982.
- [25] Zeyinzi Jiang, Zhen Han, Chaojie Mao, Jingfeng Zhang, Yulin Pan, and Yu Liu. Vace: All-in-one video creation and editing. *arXiv preprint arXiv:2503.07598*, 2025. Mask-conditioned controllable video generation; the project’s stylizer.
- [26] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. In *ICML*, 2024. arXiv:2312.04511. Plans a DAG of function calls, dispatches independent calls in parallel.
- [27] Daeun Lee, Jaehong Yoon, Jaemin Cho, and Mohit Bansal. Self-correcting text-to-video generation with misalignment detection and localized refinement. In *Findings of ACL*, pages 36464–36489, 2026. doi: 10.18653/v1/2026.findings-acl.1817.
- [28] Kelian Li, Yansong Li, Hongze Shen, Mengdi Liu, Hong Chang, and Shiguang Shan. Lenswalk: Agentic video understanding by planning how you see in videos. In *CVPR*, pages 19518–19528, 2026. arXiv:2603.24558.
- [29] Zhiqiu Lin, Deepak Pathak, Baiqi Li, Jiayao Li, Xide Xia, Graham Neubig, Pengchuan Zhang, and Deva Ramanan. Evaluating text-to-visual generation with image-to-text generation. In *ECCV*, 2024. VQAScore. arXiv:2404.01291.
- [30] Fangfu Liu, Hanyang Wang, Yimo Cai, Kaiyan Zhang, Xiaohang Zhan, and Yueqi Duan. Video-t1: Test-time scaling for video generation. In *ICCV*, pages 18671–18681, 2025.
- [31] Shaowei Liu, Zhongzheng Ren, Saurabh Gupta, and Shenlong Wang. Physgen: Rigid-body physics-grounded image-to-video generation. In *ECCV*, pages 360–378, 2024.
- [32] Yaofang Liu, Xiaodong Cun, Xuebo Liu, Xintao Wang, Yong Zhang, Haoxin Chen, Yang Liu, Tiejong Zeng, Raymond Chan, and Ying Shan. EvalCrafter: Benchmarking and evaluating large video generation models. In *CVPR*, 2024. arXiv:2310.11440.
- [33] Yuanxin Liu, Lei Li, Shuhuai Ren, Rundong Gao, Shicheng Li, Sishuo Chen, Xu Sun, and Lu Hou. FETV: A benchmark for fine-grained evaluation of open-domain text-to-video generation. In *NeurIPS*, 2023. arXiv:2311.01813.
- [34] Zhaoyang Liu, Zeqiang Lai, Zhangwei Gao, Erfei Cui, Ziheng Li, et al. Controllm: Augment language models with tools by searching on graphs. *arXiv preprint arXiv:2310.17796*, 2023. Tool use as search over a tool graph (“Thoughts-on-Graph”).
- [35] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2022.
- [36] Fanqing Meng, Jiaqi Liao, Xinyu Tan, Wenqi Shao, Quanfeng Lu, Kaipeng Zhang, Yu Cheng, Dianqi Li, Yu Qiao, and Ping Luo. Towards world simulator: Crafting physical commonsense-based benchmark for video generation. *arXiv preprint arXiv:2410.05363*, 2024. The PhyGenBench physical-commonsense generation benchmark.

- [37] Sachit Menon, Ahmet Iscen, Arsha Nagrani, Tobias Weyand, Carl Vondrick, and Cordelia Schmid. Caviar: Critic-augmented video agentic reasoning. *arXiv preprint arXiv:2509.07680*, 2025.
- [38] Juhong Min, Shyamal Buch, Arsha Nagrani, Minsu Cho, and Cordelia Schmid. Morevqa: Exploring modular reasoning models for video question answering. In *CVPR*, 2024. arXiv:2404.06511.
- [39] Antonio Montanaro, Luca Savant Aira, Emanuele Aiello, Diego Valsesia, and Enrico Magli. Motioncraft: Physics-based zero-shot video generation. In *NeurIPS*, 2024.
- [40] Saman Motamed, Laura Culp, Kevin Swersky, Priyank Jaini, and Robert Geirhos. Do generative video models understand physical principles? *arXiv preprint arXiv:2501.09038*, 2025. The Physics-IQ benchmark; visual realism does not imply physical understanding.
- [41] Sriram Narayanan, Ziyu Jiang, Srinivasa Narasimhan, and Manmohan Chandraker. Phyco: Learning controllable physical priors for generative motion. In *CVPR*, pages 41892–41902, 2026. arXiv:2604.28169.
- [42] Qwen Team. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. The vision-language reasoner (Qwen3-VL-30B-A3B-Instruct) at the centre of the critic.
- [43] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhole. Efficient and extensible algorithms for multi query optimization. In *SIGMOD*, 2000. Practical shared-subexpression MQO in a Volcano-style optimizer.
- [44] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*, 2023. arXiv:2302.04761.
- [45] Timos K. Sellis. Multiple-query optimization. *ACM Transactions on Database Systems (TODS)*, 13(1):23–52, 1988. Foundational MQO: optimize a batch of queries jointly via shared sub-expressions.
- [46] S P Sharan, Minkyu Choi, Sahil Shah, Harsh Goel, Mohammad Omama, and Sandeep Chinchali. Neuro-symbolic evaluation of text-to-video models using formal verification. In *CVPR*, pages 8395–8405, 2025. arXiv:2411.16718.
- [47] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. In *NeurIPS*, 2023. arXiv:2303.17580.
- [48] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023. arXiv:2303.11366.
- [49] Sanjay Subramanian, Medhini Narasimhan, Kushal Khangaonkar, Kevin Yang, Arsha Nagrani, Cordelia Schmid, Andy Zeng, Trevor Darrell, and Dan Klein. Modular visual question answering via code generation. In *ACL*, 2023. arXiv:2306.05392.
- [50] Kaiyue Sun, Kaiyi Huang, Xian Liu, Yue Wu, Zihan Xu, Zhenguo Li, and Xihui Liu. T2V-CompBench: A comprehensive benchmark for compositional text-to-video generation. *arXiv preprint arXiv:2407.14505*, 2024.
- [51] Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In *ICCV*, 2023. arXiv:2303.08128.
- [52] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [53] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. The Wan 2.1 open video-diffusion backbone.
- [54] Xiang Wang, Hangjie Yuan, Shiwei Zhang, Dayou Chen, Jiuniu Wang, Yingya Zhang, Yujun Shen, Deli Zhao, and Jingren Zhou. VideoComposer: Compositional video synthesis with motion controllability. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [55] Xiaohan Wang, Yuhui Zhang, Orr Zohar, and Serena Yeung-Levy. Videoagent: Long-form video understanding with large language model as agent. In *ECCV*, 2024. arXiv:2403.10517.
- [56] Ziyang Wang, Shoubin Yu, Elias Stengel-Eskin, Jaehong Yoon, Feng Cheng, Gedas Bertasius, and Mohit Bansal. Videotree: Adaptive tree-based video representation for llm reasoning on long videos. In *CVPR*, 2025. arXiv:2405.19209.
- [57] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023. Plan-up-front, then execute; the ReWOO plan-execute separation.
- [58] Qiyao Xue, Xiangyu Yin, Boyuan Yang, and Wei Gao. Phyt2v: Llm-guided iterative self-refinement for physics-grounded text-to-video generation. In *CVPR*, pages 18826–18836, 2025. arXiv:2412.00596.
- [59] Rui Yang, Lin Song, Yanwei Li, Sijie Zhao, Yixiao Ge, Xiu Li, and Ying Shan. Gpt4tools: Teaching large language model to use tools via self-instruction. In *NeurIPS*, 2023. arXiv:2305.18752.
- [60] Xindi Yang, Baolu Li, Yiming Zhang, Zhenfei Yin, Lei Bai, Liqian Ma, Zhiyong Wang, Jianfei Cai, Tien-Tsin Wong, Huchuan Lu, and Xu Jia. Vlpp: Towards physically plausible video generation with vision and language informed physical prior. In *ICCV*, pages 12360–12370, 2025. arXiv:2503.23368.

- [61] Yuhang Yang, Ke Fan, Shangkun Sun, Hongxiang Li, Ailing Zeng, FeiLin Han, Wei Zhai, Wei Liu, Yang Cao, and Zheng-Jun Zha. Videogen-eval: Agent-based system for video generation evaluation. *arXiv preprint arXiv:2503.23452*, 2025.
- [62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023. arXiv:2210.03629.
- [63] Ke Zhang, Cihan Xiao, Jiacong Xu, Yiqun Mei, and Vishal M. Patel. Think before you diffuse: Infusing physical rules into video diffusion. *arXiv preprint arXiv:2505.21653*, 2025. The parent “DiffPhy” work this project extends. Title as verified live on the arXiv abstract page; some project docs cite an earlier subtitle “Physics-Aware Video Generation via Chain-of-Thought Conditioning.”.
- [64] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [65] Qin Zhang, Peiyu Jing, Hong-Xing Yu, Fangqiang Ding, Fan Nie, Weimin Wang, Yilun Du, James Zou, Jiajun Wu, and Bing Shuai. Physion-eval: Evaluating physical realism in generated video via human reasoning. *arXiv preprint arXiv:2603.19607*, 2026.
- [66] Yongdong Zhang, Yi Wang, Yixiao Ge, Yuying Ge, Xintao Li, Ying Shan, and Xiaohu Wang. Timelens: Rethinking video temporal grounding with multimodal llms. *arXiv preprint arXiv:2512.14698*, 2025. Closest published reference for the deployed action arm (the MCG-NJU TimeLens2-8B checkpoint has no standalone paper).
- [67] Artem Zhohus, Carl Doersch, Yi Yang, Skanda Koppula, Viorica Patraucean, Xu He, Ignacio Rocco, Mehdi S. M. Sajjadi, Sarath Chandar, and Ross Goroshin. Tapnext: Tracking any point (tap) as next token prediction. *arXiv preprint arXiv:2504.05579*, 2025. The point-tracking backbone behind the kinematic (physics) arm’s TAPNext++.
- [68] Yuchen Zhuang, Xiang Chen, Tong Yu, Saayan Mitra, Victor Bursztyn, Ryan A. Rossi, Somdeb Sarkhel, and Chao Zhang. Toolchain*: Efficient action space navigation in large language models with a* search. In *ICLR*, 2024. arXiv:2310.13227.